

Université Évry Paris-Saclay

Mention : Ingénierie des Systèmes Complexes

Parcours : M2 Robotique Industrielle

Master M2 Robotique Industrielle FI

Année universitaire

2025–2026

*From a Multi-Robot Digital Twin to Safety-Gated
Neuro-Symbolic Language-to-Motion Control*

Université de Toulouse

Maison de la Formation Jacqueline Auriol (MFJA)

NOM : IBRAHIM

Prénom : Ali

Numéro d'étudiant : 20255709

Statut : Formation initiale

Tuteurs de stage : Olivier STASSE et Diane BURY

Identification Page

Student

Name	Ali IBRAHIM
Programme	M2 ISC — Industrial Robotics track
Academic year	2025–2026
Student number	20255709
Telephone	07 50 79 35 89
Email	ali.ibrahim@etud.univ-evry.fr

Host organisation

Organisation	Université de Toulouse
Host service	Maison de la Formation Jacqueline Auriol (MFJA)
Address	1 rue Tarfaya, 31400 Toulouse, France
Telephone	05 62 25 88 80

Host supervisors

Olivier STASSE	Scientific supervisor; olivier.stasse@laas.fr Telephone: 05 61 33 79 82
Diane BURY	Scientific supervisor; diane.bury@utoulouse.fr Telephone: 05 62 25 88 18

Academic supervisor

Name	Nicolas SEGUY
Role	Academic supervisor
Email	nicolas.seguy@univ-evry.fr

Mission

Period	2 March 2026 – 28 August 2026
Host unit	Robotics platforms, MFJA
Official title	Development of a unified interface for industrial and mobile robots in simulation
Report language	English

Abstract

The internship developed a common simulation environment for the industrial and mobile robots used at the Maison de la Formation Jacqueline Auriol (MFJA). The initial assignment was to model Room 315 in Gazebo and provide consistent Robot Operating System 2 (ROS 2) access to its robots and transport system. Completing this stage ahead of schedule opened an investigation inspired by vision–language–action (VLA) research, based on overhead-camera perception and natural-language interaction. The implemented architecture is a schema-constrained neuro-symbolic language-and-vision interface rather than an end-to-end VLA policy.

The resulting system combines configuration-driven robot launch, calibrated rail geometry, continuous multi-shuttle motion and typed ROS 2 command and state interfaces. A locally deployed Qwen2.5-1.5B-Instruct model converts an English request into a structured task proposal, while a paired-view ResNet-18 estimates shuttle state from the simulated cameras. Schema and domain checks are applied before PDDL/PlanSys2 produces a plan, and the resulting motion is supervised and confirmed from the observed system state.

The English-to-motion workflow completed 12/12 isolated cold-start runs: six on each rail, covering all eight shuttle identities and five planned case families. Every run reached its expected terminal state. A separate ten-case language evaluation matched the expected result in every case, including clarification, rejection and cancellation. On the locked 256-scenario Test set, the configured visual checkpoint achieved 100% rail-side accuracy, 99.2537% loaded-state accuracy and 65.2985% exact-block accuracy.

All implementation and evaluation took place in Gazebo simulation.

Keywords: ROS 2; Gazebo Harmonic; industrial robotics; digital twin; multi-robot systems; flexible manufacturing; PDDL; PlanSys2; computer vision; language-and-vision interface; neuro-symbolic control; safety supervision.

Résumé

Le stage portait sur une interface unifiée pour les robots industriels et mobiles en simulation, destinée à l'enseignement, à l'intégration logicielle et à la recherche. Le premier lot de travail a consisté à modéliser la salle 315 de la Maison de la Formation Jacqueline Auriol (MFJA) dans Gazebo et à produire une simulation exploitable de ses robots et de son système de transport. Son achèvement anticipé a permis d'étendre le projet à une étude fondée sur des caméras en hauteur et une approche vision-langage-action (VLA). L'objectif d'ingénierie est ainsi devenu l'intégration de robots hétérogènes, du transport flexible, de l'état visuel et des requêtes en langage naturel, tout en maintenant un contrôle déterministe de l'actionnement.

Le développement a suivi une démarche incrémentale. Chaque couche s'appuie sur une représentation de référence et une interface ROS 2 limitée, validées avant l'intégration de la couche suivante. Le travail a produit quatre contributions liées: un socle de jumeau numérique simulé avec lancement piloté par configuration; des graphes ferroviaires calibrés et un mouvement continu multi-navette supervisé; des contrats commande/état typés avec supervision déterministe et vérification des effets; enfin, une boucle neuro-symbolique dotée de deux interfaces apprises limitées. Un modèle Qwen2.5-1.5B-Instruct déployé localement propose une représentation sémantique soumise à un schéma strict, tandis qu'un ResNet-18 à deux vues, entraîné sur le corpus généré de la salle 315, estime des valeurs d'état structurées. La fusion et la validation de domaine déterministes créent la tâche acceptée. PDDL/PlanSys2, la traduction du plan, la supervision et la vérification des effets conservent l'autorité de décision et d'exécution.

Dans le périmètre simulé de la salle 315, la campagne intégrée allant de la requête en anglais au mouvement a mené à bien les douze exécutions isolées démarrées à froid. Elle comprenait six exécutions par rail, couvrait les huit identités de navette et cinq familles de cas prédéclarées ; le résultat terminal déclaré a été vérifié dans chaque cas. Une évaluation distincte et figée du contrat linguistique sur dix cas a reproduit chaque résultat final déclaré sans aucune résolution automatique non sûre. Le checkpoint visuel utilisé pour l'évaluation est identifié par hachage. Il a atteint 100 % de précision du côté de rail, 99,2537 % de précision de l'état de charge et 65,2985 % de précision pour la classification exacte du bloc lors d'une unique évaluation sur l'ensemble Test verrouillé de 256 scénarios.

La mise en œuvre et son évaluation restent limitées à la simulation. La configuration actuelle utilise le checkpoint visuel identifié. Le projet fournit à la MFJA une base réutilisable pour l'enseignement et la recherche, des contrats d'intégration explicites et un socle traçable pour poursuivre les développements et les expérimentations en simulation.

Mots-clés : ROS 2 ; Gazebo ; robotique industrielle ; jumeau numérique ; système multi-robot ; PDDL ; PlanSys2 ; vision ; VLA ; commande neuro-symbolique ; supervision de sécurité.

Acknowledgements

I would like to sincerely thank Olivier Stasse and Diane Bury for their scientific guidance throughout this internship. Our weekly discussions and their constructive feedback helped me refine the technical architecture, improve the experimental methodology and develop the initial modelling assignment into a broader integration and research project for the MFJA platforms.

I would also like to thank Nicolas Seguy for his academic supervision and for following the progress of the placement. Finally, I am grateful to the administrative and technical staff at the MFJA for coordinating access to the facilities, helping with organisational procedures and providing the practical support needed throughout the internship.

Use of Generative Artificial Intelligence

Generative artificial intelligence tools, primarily OpenAI ChatGPT, were used during the preparation of this work as supporting tools. Their use included improving the grammatical correctness, clarity, and fluency of the report; enhancing the visual quality and presentation of selected figures and diagrams; and assisting with the generation, organization, and refactoring of selected portions of code, as well as with repetitive development tasks and workflow optimization.

All technical choices, system design, implementation, experiments, results, analyses, and conclusions presented in this report remain the author's own work and responsibility. AI-assisted code, figures, and suggestions were systematically reviewed, adapted where necessary, tested or validated as appropriate, and integrated by the author. Generative AI was therefore used as a supporting tool to improve efficiency and presentation quality, rather than as a substitute for the author's technical reasoning, engineering decisions, implementation, or validation. This disclosure follows the university reporting guidance [20].

Glossary and Acronyms

Acronyms

APE	Activité Principale Exercée, the French principal-activity code
CAD	Computer-Aided Design
CLI	Command-Line Interface
CSV	Comma-Separated Values
DDS	Data Distribution Service
GUI	Graphical User Interface
JSON	JavaScript Object Notation
MFJA	Maison de la Formation Jacqueline Auriol
NAF	Nomenclature d'Activités Française
PDDL	Planning Domain Definition Language
PLC	Programmable Logic Controller
POPF	Partial Order Planning Forwards
QoS	Quality of Service
RGB	Red–Green–Blue colour representation
ROS	Robot Operating System
SDF	Simulation Description Format
STL	Stereolithography mesh format
TF	ROS transform tree
URDF	Unified Robot Description Format
VLA	Vision–Language–Action
YAML	YAML Ain't Markup Language

Project-specific terms

- Digital twin** A versioned digital representation of a defined physical system that includes its relevant information flows.
- Fail-closed** A policy that withholds an operation when required state is missing, stale, contradictory or from an unapproved source.
- ObservedState** A versioned collection of current observed facts and their provenance at one state identifier.
- SHA-256** A cryptographic digest used here to identify the exact files or model weights associated with an evaluation.
- TaskGoal** The fixed task description created after rule-based validation and operator confirmation of a semantic draft.

Contents

1	General Introduction	13
2	Host Organisation and MFJA Robotics Platforms	14
2.1	Université de Toulouse	14
2.2	Maison de la Formation Jacqueline Auriol	14
2.3	The Room 315 robotics environment	14
2.4	Organisation and responsibilities	15
2.5	Position of the mission within the host	15
2.6	Economic and strategic value	16
3	Mission, Project Management and Personal Contribution	17
3.1	Mission and work packages	17
3.2	Chronology and decision gates	17
3.3	Decision progression	17
3.4	Requirements and review process	20
3.5	Personal responsibility, collaboration and resources	20
3.6	Autonomy and reflective engineering examples	20
3.7	Competencies and professional learning	21
4	Technical Background and Design Choices	22
4.1	ROS 2 as the integration layer	22
4.2	Gazebo Harmonic and description formats	22
4.3	Coordinate and rail representations	22
4.4	Symbolic task planning	23
4.5	Architectural alternatives and final design	24
4.6	Visual-state model	24
5	Building the MFJA Digital Twin	25
5.1	Reconstruction strategy	25
5.2	World decomposition	25
5.3	Repository refactoring	25
5.4	Model integration	27
5.5	Launch architecture	28
5.6	Operational profiles	28
5.7	Build and environment reproducibility	28
5.8	Outcome and model fidelity	28
6	Unified Multi-Robot Access and Control	29
6.1	What “unified” means	29
6.2	Multi-robot operation in one Gazebo process	29
6.3	Configuration-driven robot registry	29
6.4	Dynamic bridge generation	29
6.5	Joint-command helper	30
6.6	Interface portability	30
6.7	Results	32
7	Room 315 Kinematic Transport System	33
7.1	Modelling decision: dynamic or kinematic	33

7.2	Kinematic transport backend	33
7.3	From SolidWorks points to a continuous rail graph	33
7.4	Topology and switch assignments	35
7.5	Devices and public naming	36
7.6	Multi-shuttle identity and lifecycle	38
7.7	Motion, slots and occupancy	38
7.8	Operator-facing observation and maintenance	40
7.9	Headway and collision constraints	40
7.10	Interior staging and dense blocker cases	40
7.11	Validation of the transport model	40
8	Typed Interfaces, Contracts and Safety Supervision	41
8.1	A typed command and state interface	41
8.2	Command/state separation	41
8.3	Sparse partial-update semantics	41
8.4	Versioned contracts	43
8.5	Observed-state semantics	43
8.6	Supervisor responsibilities	44
8.7	Reservations and deadlock	45
8.8	Source of switch feedback	45
8.9	Checking command effects	45
8.10	Manual and emergency paths	45
9	AI Inputs: English and Visual-State Learning	47
9.1	Phase 2 objective and implementation sequence	47
9.2	Reused components and project-specific development	47
9.3	Building the English task interface	47
9.4	Generating the Room 315 visual corpus	48
9.5	Designing and training the visual-state estimator	49
9.6	Integration outputs	51
10	From an English Instruction to Shuttle Motion	53
10.1	Runtime architecture	53
10.2	Step 1—interpret and confirm the English task	55
10.3	Step 2—read the current scene and ground the goal	55
10.4	Step 3—build a fresh PDDL problem and plan	55
10.5	Step 4—translate, supervise and move	56
10.6	Step 5—check the result and decide what follows	57
10.7	Case B03 from instruction to arrival	57
11	Experiments and Results	58
11.1	Integrated English-to-motion campaign	58
11.2	Visual model selection and locked-Test result	60
11.3	Automated software checks	61
11.4	Review of the internship objectives	62
12	Limitations and Perspectives	63
12.1	Current limitations	63
12.2	Next evaluation steps	63
12.3	Continued use at MFJA	64
12.4	Focused research extensions	64
13	Conclusion	65

Bibliography	66
A Reproducibility Records	68
A.1 Software environment and handover	68
A.2 Evaluation files and identifiers	68
B Capability and Interface Reference	71
B.1 Operator capability index	71
B.2 Public rail interfaces	71
B.3 Canonical assets	72
B.4 Runtime responsibility summary	73

List of Figures

2.1	Gazebo overview of Room 315 and its four robot workstations.	15
2.2	Organisation of the internship and distribution of responsibilities.	15
3.1	Engineering, reporting and defence-preparation chronology.	18
3.2	Dependency-driven progression of the two project phases.	19
4.1	Geometry and topology as complementary rail representations.	23
5.1	Reconstruction progression from the floor model to Room 315.	26
5.2	ROS 2 package ownership and launch-composition dependencies.	27
6.1	Configuration-driven multi-robot command and feedback isolation.	31
7.1	Ordered CSV source points and current Hermite rail geometry.	35
7.2	Public A2–A23–A3 transition rule.	36
7.3	Current right-rail sensors, stopper identities and slot numbering.	37
7.4	Paired Gazebo camera views of the complete eight-shuttle fleet.	39
8.1	Supervised sparse commands and controller feedback.	42
8.2	Eligibility gates for facts entering planner state.	44
9.1	Paired Room 315 camera observation used for visual training.	49
9.2	Paired-view visual model and fixed-order state decoding.	50
9.3	Visual-data, training and checkpoint-promotion sequence.	52
10.1	Current Room 315 English-to-motion architecture.	54
11.1	Initial and final right-camera frames for campaign case B03.	59
11.2	Outcomes and coverage of the twelve-run campaign.	60
11.3	Evaluation layers and the questions they address.	62

List of Tables

3.1	Engineering competencies and concrete project examples.	21
4.1	Representative architectures compared by the roles assigned to learning, planning and command execution.	24
6.1	Principal robot selectors and command surfaces.	30
7.1	Public successor map shared by both Room 315 rails.	35
8.1	Principal closed-loop contracts and their producers.	43
9.1	Phase 2 development sequence.	47
9.2	Reused foundations and project-specific Phase 2 work.	48
9.3	Configuration of the selected visual training run.	51
10.1	Principal symbolic decision cases in the current Room 315 PDDL domain.	56
10.2	English-to-motion record for campaign case B03.	57
11.1	Campaign rows and recorded terminal results.	58
11.2	Locked-Test metrics for the configured visual-state checkpoint.	60
11.3	Assessment of the four internship objectives.	62
12.1	Priority experiments for extending the current evaluation.	64
A.1	Files and identifiers for the reported results.	69
B.1	Current capabilities represented in the operator runbook.	71
B.2	Condensed rail-interface reference.	72
B.3	Canonical Room 315 mapping.	72
B.4	Responsibilities in the Room 315 task workflow.	73

Chapter 1

General Introduction

Context and engineering objective. The MFJA robotics platforms bring together industrial manipulators, a mobile robot, transport cells, cameras and programmable devices. Integrating this equipment in simulation requires compatible models, coordinate frames, namespaces, commands and state descriptions. The challenge was therefore broader than constructing a visually accurate Gazebo scene: the simulated equipment also had to behave consistently enough to support routing, planning and experimentation.

The project then examined whether camera observations and English requests could describe a task without sending motor commands directly. The language and vision paths supply information about the request and the scene. PlanSys2 produces the symbolic sequence, and the supervisor checks each requested step before execution. Neither learned model produces a plan or an action vector.

Objectives and principal contribution. The project pursued four engineering objectives within simulation:

- O1.** establish a maintainable Gazebo representation of the MFJA third floor and Room 315, with configuration-driven launch and unified ROS 2 access to the supported simulated robots;
- O2.** implement continuous multi-shuttle transport over calibrated rail geometry and explicit topology, including device semantics, routing, reservations and collision-avoidance constraints;
- O3.** define typed command and state interfaces, together with supervision rules for checking commands and confirming their effects; and
- O4.** design and evaluate a neuro-symbolic workflow that combines natural-language goal interpretation and paired-camera state estimation with PDDL/PlanSys2 planning and supervised execution.

The four objectives form a progression from the simulated environment and its transport system to task-level interaction through language and vision. The objective identifiers O1–O4 are used again in table 11.3 to connect each objective to its evaluated result.

Method and headline results. Development followed short build, test and review cycles. Each part was first implemented and checked separately, then connected to the preceding layers. This made it possible to resolve modelling, interface and coordination issues before evaluating the complete workflow. The software, dataset and model versions used for the reported evaluations are listed in appendix A.

The complete workflow and the visual model were evaluated separately. Their results and the limits of each evaluation are presented in chapter 11.

Report structure. The host environment, mission and principal design choices are introduced in chapters 2 to 4. The engineering foundations are then developed in chapters 5 to 8, followed by the language integration, paired-camera data generation and visual training process in chapter 9. Chapter 10 follows one English instruction through planning, supervised motion and result checking, and chapters 11 and 12 present the results and future work.

Chapter 2

Host Organisation and MFJA Robotics Platforms

2.1 Université de Toulouse

The internship was formally hosted by Université de Toulouse and carried out at the MFJA site, 1 rue Tarfaya, Toulouse. As a public university, the institution combines higher education, research and knowledge transfer. Its official Nomenclature d'activités française/Activité principale exercée (NAF/APE) code is 85.42Z [17], which INSEE classifies as *higher education*, including first-, second- and third-cycle university programmes [7].

Université de Toulouse has operated as an experimental public institution since January 2025. Official figures published for 2026 indicate approximately 39,000 students and 4,200 staff [19]. These figures concern the university as a whole, while the internship took place within the smaller MFJA environment.

2.2 Maison de la Formation Jacqueline Auriol

The MFJA is a shared centre for engineering education on the Toulouse Aerospace innovation campus. Its activities cover research and innovation, knowledge and skills transfer, and initial and continuing education. According to its official presentation, the site brings together more than 1,500 students from 15 mechanical-engineering programmes associated with Université de Toulouse, INSA and ISAE. It supports work in mechanical engineering, manufacturing, aeronautics, automation and robotics in collaboration with industrial partners and research laboratories [18].

Because the facilities are shared across teaching and research activities, the software developed for the platform must support several configurations, remain usable by different operators and be maintainable beyond a single development environment. These requirements influenced the package and launch architecture presented in Chapters 5–6.

2.3 The Room 315 robotics environment

Room 315 contains two mirrored flexible transport cells, each with four work positions associated with industrial robots. In the configuration used for this project, the right rail serves Yaskawa HC10DT positions 1–2 and Stäubli TX2 positions 3–4, while the left rail serves Yaskawa HC10 positions 1–2 and KUKA KR6 positions 3–4. Each circuit contains outer and inner branches, four switching zones, indexing detectors and stoppers. A shuttle may be empty or carry a payload.

This arrangement brings together several engineering disciplines:

- mechanical layout, computer-aided design (CAD), kinematics and collision geometry;
- industrial automation, sensors, stoppers and route switching;
- robot middleware, namespacing, timing and distributed interfaces;
- planning, scheduling and multi-agent resource conflicts;
- computer vision, data collection and model evaluation;
- human–robot interaction through task-level language.

Together, these disciplines make Room 315 a practical platform for studying the integration of robotics, transport, perception and control. Figure 2.1 shows the simulated layout; Chapters 7–8 define the rail topology, device semantics and supervisory constraints.

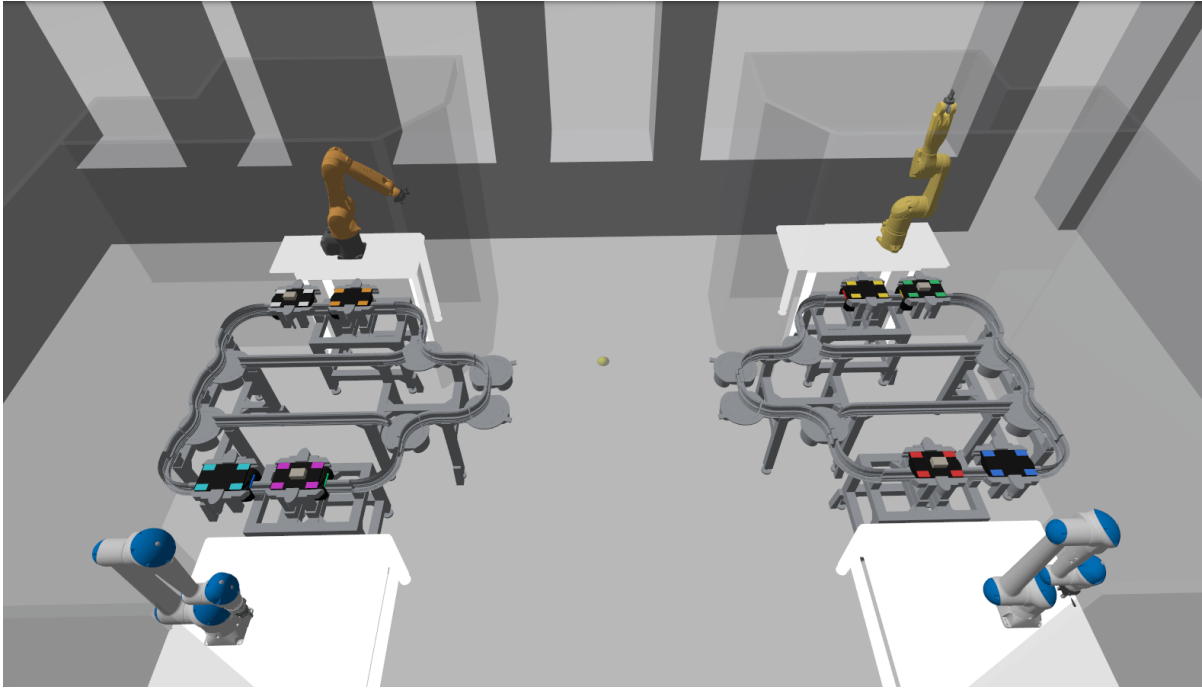


Figure 2.1: Gazebo view of Room 315. Four industrial-robot workstations surround two mirrored rail-transport cells containing eight stopped shuttles. The measured rail geometry and device locations are presented in chapter 7.

2.4 Organisation and responsibilities

Figure 2.2 presents the supervision and responsibilities directly associated with the internship.

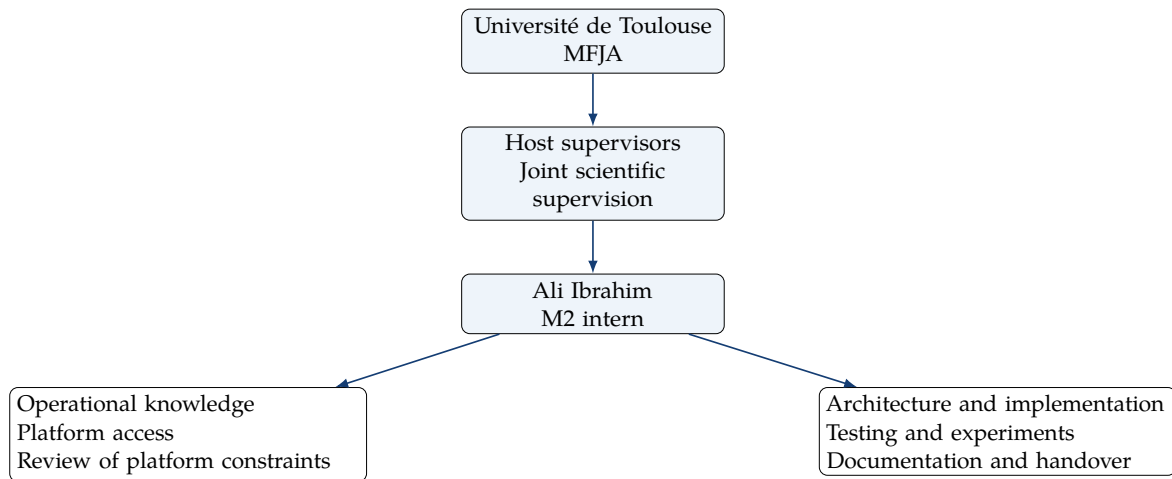


Figure 2.2: Organisation of the internship and distribution of responsibilities.

The host supervisors provided joint scientific direction and reviewed choices related to the platform. MFJA personnel supported access, scheduling and practical organisation. I was responsible for the architecture, implementation, evaluation, documentation and handover described in chapter 3.

2.5 Position of the mission within the host

The mission connects platform operation with research prototyping. Platform managers, students and researchers can use the system to launch robots and rail scenarios reproducibly. The same foundation can support synthetic-data generation, task-planning experiments and comparisons

between simulation configurations. Possible uses include a teaching exercise on ROS 2 topics, a planner benchmark, a visual-perception dataset or a simulation demonstrator.

This range of uses motivated the emphasis on reusable configuration, explicit interfaces and documented operating procedures.

2.6 Economic and strategic value

Because MFJA is a public teaching and research platform, the mission's value is assessed through practical reuse, training capacity and reduced integration effort. The main benefits are:

- **reuse:** one model and launch framework avoids recreating an integration for each practical class or research project;
- **risk reduction:** simulation and supervised execution check naming, topology and commands before access to costly equipment;
- **asset visibility:** a digital twin makes robot capabilities available for remote preparation, including capabilities that might otherwise remain under-used;
- **training capacity:** repeatable scenario generation creates data and exercises without monopolising the physical cell;
- **maintenance:** typed interfaces, tests and documentation reduce dependence on knowledge held by a single developer;
- **technology transfer:** the MFJA can demonstrate current ROS 2, planning and AI methods while maintaining clear safety boundaries.

These benefits align with the MFJA's institutional role of sharing costly engineering resources and connecting education with contemporary industrial methods.

Chapter 3

Mission, Project Management and Personal Contribution

3.1 Mission and work packages

The internship mission focused on unified ROS 2 access to industrial and mobile robots in simulation [21]. The first practical assignment was more specific: model Room 315 in Gazebo and establish a usable simulation environment for its robotic and transport equipment. This objective was completed earlier than expected, and the supervisors proposed an additional research direction combining overhead-camera perception with natural-language task interaction. The internship dates were unchanged.

The implemented work is organised into two phases:

Phase 1—initial practical work package Room 315 Gazebo modelling and a reusable, selectively launched multi-robot and rail simulation foundation.

Phase 2—additional research direction Paired cameras, visual-state estimation, validated language goals, PDDL/PlanSys2 and safety-gated neuro-symbolic control.

The phases describe the project chronology, whereas objectives O1–O4 in chapter 1 define the units used to assess the completed work. The initial phase established the simulation, multi-robot and transport foundations. The additional research phase extended the typed interfaces and connected visual and language inputs to planning, supervised execution and effect verification.

3.2 Chronology and decision gates

The internship schedule was fixed from the outset. As shown in figure 3.1, the technical phases overlap because integration, testing and correction continued after each main decision gate.

3.3 Decision progression

Figure 3.2 summarises how the project progressed from the initial simulation to the integrated neuro-symbolic system. Each stage depended on working results from the preceding stage.

Major technical decisions were taken after weekly discussions with the supervisors. The agreed solutions were then implemented and checked through configuration, tests and integration runs.

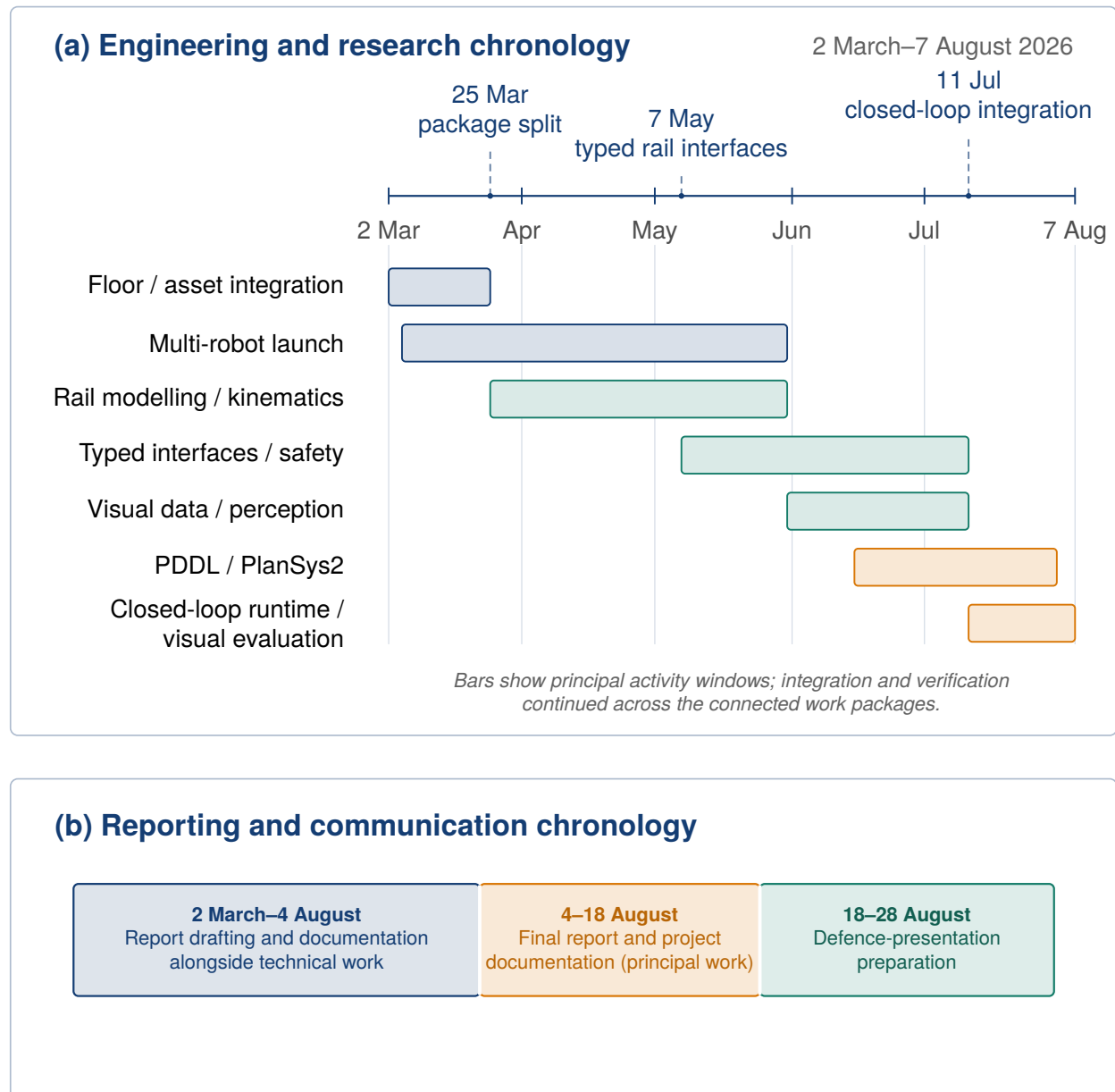


Figure 3.1: Internship chronology. Engineering, integration and final verification run through 7 August in panel (a), overlapping the report-focused period that began on 4 August. Documentation continues to 18 August; the final period, 18–28 August, is reserved for defence-presentation preparation.

Phase 1 — initial assigned task

1. World and robots

Engineering need: unify heterogeneous assets, names and topics. **Implementation:** correct the CAD scene, separate package ownership and launch from a namespaced registry. **Result:** select any supported simulated-robot subset.



2. Shuttle motion

Engineering need: repeatable motion through switch transitions. **Implementation:** use arc-length Hermite paths and Gazebo pose control. **Result:** repeatable continuous motion with an explicit kinematic model.



3. Topology and devices

Engineering need: encode legal routes, stops and device semantics. **Implementation:** declare directed segments, switches, slots, stoppers and sensors in YAML. **Result:** planner-readable dual-rail state and coordination.

Phase 2 — additional research direction

4. Visual and task interfaces

Engineering need: interpret English tasks and the current camera scene. **Implementation:** pair overhead RGB views, keep labels separate from model input, and convert semantic proposals into confirmed task goals. **Result:** structured language and visual inputs for planning.



5. Planning and supervised execution

Engineering need: convert the current task and state into supervised motion. **Implementation:** build a fresh PDDL problem, select one symbolic step, translate it, pass it through the supervisor and check the resulting state. **Result:** one-step planning and execution that repeats until completion.



6. Experiments and results

Engineering need: measure perception and confirm complete task behaviour. **Implementation:** use grouped visual splits, model metrics, software tests and a 12-run cold-start operator-interface campaign. **Result:** model measurements and complete positive runs across both rails, all eight identities and five case families.

Figure 3.2: Project progression as a dependency chain. Each stage converts a technical objective into an engineering result required by the next stage.

3.4 Requirements and review process

Requirements were expressed at three levels. Launch arguments, topics and runbooks define how operators use the system. Executable contracts and tests specify what each component must accept, reject or preserve. Artefact manifests and hashes give datasets and models reproducible identities. The main requirements across these levels were robot isolation, stable shuttle identity, partial device updates without unintended changes, current state before planning, fixed translation rules and a new observation after each executed plan step.

Each property was checked where it was implemented: geometry and coordinate frames, interface semantics, shared-rail coordination and learned-model output. The safeguards and data-leakage controls are detailed in chapters 7 to 9.

The agreed solutions were reflected in configuration and version-controlled code, then exercised through the corresponding tests and experiments. The identifiers for the relevant sources, datasets and models are listed in appendix A; the corresponding objective-to-result assessment is reported in table 11.3.

3.5 Personal responsibility, collaboration and resources

Within the shared repository, I was responsible for the third-floor and Room 315 models; the selective multi-robot launch architecture; calibrated rail transport, device logic and typed interfaces; task contracts, PDDL/PlanSys2 integration and supervised one-step execution; the paired-camera dataset and visual training pipeline; and the tests, manifests, runbooks and handover documentation.

The supervisors provided scientific direction, technical review and knowledge of the platform. I used the weekly discussions to present alternatives and the information available for decisions that affected shared interfaces, software safeguards or experimental claims. Once a decision had been agreed, I was responsible for implementing it consistently in the relevant configuration, code, tests and documentation.

In practice, resource management concerned the fixed schedule, shared platform access, simulation and training time, dataset and checkpoint storage, and the testing workload. I kept compute-intensive perception and language nodes optional, prioritised reusable configuration and regression tests, and prepared build and run instructions so that the host could continue using the work.

3.6 Autonomy and reflective engineering examples

I divided the broad integration objective into testable increments and kept early implementation choices reversible. I retained a design only after its interfaces and implementation were consistent and the corresponding regression tests passed. I brought decisions affecting shared interfaces, experimental claims or software safeguards to the supervisors for review before finalising them. The following two examples show how this process worked in practice.

3.6.1 Selecting a defensible level of rail fidelity

At the beginning of the rail work, I compared a dynamic wheel–rail simulation with a kinematic rail-following model. The measured mechanical parameters required for a credible dynamic model were not available, and that option was expected to require more integration time and computation. The project questions instead concerned routing, sensor crossings, slot arrival, perception and supervised execution. I therefore proposed the calibrated graph and arc-length solution detailed in chapter 7. I discussed the comparison with the supervisors during a weekly review. The agreed priority was repeatable behaviour at the level required by those questions. Mechanical effects that could not be supported by the available data were left outside the model. I then implemented the model, checked the configured geometry and tested routing, sensor feedback and slot-arrival

behaviour. This decision taught me to match model fidelity to the question being studied and to treat missing data, schedule and computing capacity as engineering constraints.

3.6.2 Choosing PDDL/PlanSys2 over end-to-end VLA control

The initial plan for the second phase was to train an end-to-end vision–language–action model that would infer actions directly, without PDDL. After discussing this option with my supervisors, they sent me several papers on VLA and hybrid planning architectures. From this comparison, I proposed a different division of responsibilities: the visual model estimates the scene state from the paired camera images, while PDDL/PlanSys2 determines the action sequence, which is then executed through the supervisor and controllers. This approach was better suited to Room 315 because its rail topology and device rules are explicit, whereas the available dataset contains scene-state labels rather than action demonstrations. It also made the plans easier to inspect and execution errors easier to locate. This became the architecture implemented in chapters 9 and 10.

3.7 Competencies and professional learning

Table 3.1 relates the principal competencies developed during the internship to concrete examples from the project.

Table 3.1: Engineering competencies and concrete project examples.

Competency	Project example
Systems analysis	Separation of geometry, perception, planning, supervision and actuation-provider ownership
Robotics integration	ROS 2/Gazebo assets, controllers, launch composition, namespaces and device feedback
Algorithm design	Arc-length graph, topology-aware routing, reservations, clearance and effect checks
AI/ML method	Local language-model integration, leakage-controlled paired-image data, grouped splitting and multi-head visual training
Software quality	Typed contracts, focused regression tests, configuration manifests and reproducible result records
Project management and communication	Incremental milestones, supervisory reviews, diagrams, runbooks, result tables and explicit scope

These two decisions changed how I assessed technical alternatives. I initially associated a more detailed dynamic model or a more end-to-end learned policy with a more advanced solution. The comparisons showed me that each choice had to be justified by the available data, the behaviour required in Room 315 and the way the result could be tested. That lesson guided the later integration work.

Chapter 4

Technical Background and Design Choices

4.1 ROS 2 as the integration layer

ROS 2 provides processes (nodes), typed interfaces, discovery and distributed communication. Topics suit continuous streams, services suit short request–response operations and actions suit long-running tasks with feedback [12]. These distinctions guided the choice of typed streams, explicit request–response operations and isolated namespaces for the project. The resulting interfaces are presented in chapters 6 and 8.

ROS 2 Quality of Service (QoS) matters because camera data, state and commands have different reliability and latency needs [13]. The current design therefore combines conservative communication defaults with explicit freshness checks. Profiles for other network conditions would require measurement in their target environment.

4.2 Gazebo Harmonic and description formats

Gazebo Harmonic is the recommended Gazebo pairing for ROS 2 Jazzy on Ubuntu 24.04 [14]. The Simulation Description Format (SDF) describes the world, environment models, sensors, physics and plugins [16]; the Unified Robot Description Format (URDF) remains useful for robot kinematic trees and `robot_state_publisher`. The `ros_gz_bridge` translates selected Gazebo Transport messages to or from ROS 2 [15].

Simulation serves four complementary purposes in this project:

1. validate coordinate frames, namespaces and controller interfaces across heterogeneous robot models in a controlled environment;
2. support instruction in multi-robot launch and command workflows;
3. execute repeatable rail and safety scenarios at scale;
4. generate labelled overhead observations for perception training.

These purposes impose different fidelity requirements. High-resolution visual meshes support camera-based observation, while simplified collision geometry keeps contact handling efficient and predictable. The shuttle study requires continuous, repeatable transport rather than detailed wheel–rail contact dynamics; it therefore uses kinematic pose control. The corresponding transport assumptions are detailed in chapter 7.

ISO 23247 defines general principles and requirements for manufacturing digital-twin frameworks [8]. In this project, the digital-twin model combines a versioned representation of the simulated environment with clearly defined information flows, assumptions and boundaries. Visual similarity alone is insufficient for that purpose.

4.3 Coordinate and rail representations

A rail path is represented by piecewise geometric segments. Each segment has an arc-length coordinate s , a physical length L , and a normalised coordinate

$$\rho = \frac{s}{L}, \quad 0 \leq \rho \leq 1. \quad (4.1)$$

Using equation (4.1), the controller evaluates position and tangent from s , then constructs the shuttle pose and heading. Cubic Hermite interpolation provides continuous tangents between

calibrated source points. A separate topology graph defines which segments connect for each switch assignment. Geometry therefore determines *where the rail is*, while topology determines *which route is open*. This relationship is shown in figure 4.1.

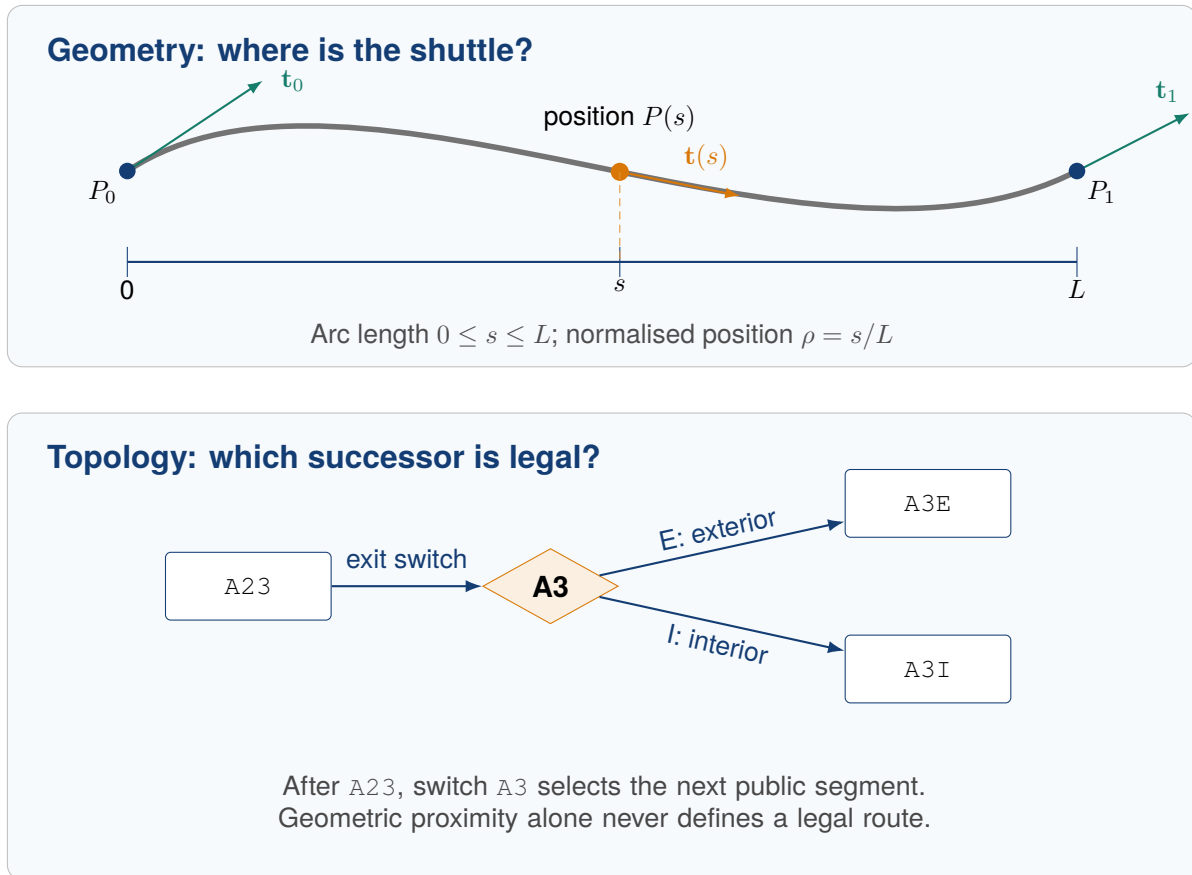


Figure 4.1: Complementary rail representations. Hermite geometry supplies position and tangent for pose construction; topology and switch state select the valid successor exposed through the public interface.

The normalised coordinate supports relative placement, while s preserves the metric distance needed for motion control. Chapter 7 presents the transport implementation, and chapters 8 to 10 define the visual-state contract and final arrival checks.

4.4 Symbolic task planning

The Planning Domain Definition Language (PDDL) describes objects, predicates, actions, preconditions and effects [11]. PlanSys2 integrates PDDL-based planning into ROS 2 and exposes domain, problem, planner and executor components [10]. The selected solver is POPF, a forward-chaining partial-order planner [2].

Symbolic planning is appropriate for Room 315 because the important decisions are discrete and relational: which shuttle is present, which slot is occupied, which branch is selected, whether a stopper is open, whether a blocker must be staged and whether the goal has been achieved. Continuous motion remains inside the controller and effect-verification layers.

The planner provides the symbolic sequence, while continuous movement is handled by the controller. The current deployment combines the PlanSys2 planner service with project-specific translation, supervision and execution components. These component responsibilities are defined in chapter 10.

4.5 Architectural alternatives and final design

The term vision–language–action covers architectures with materially different responsibility splits. RT-2 maps images and language to tokenised robot actions inside one learned policy, while TinyVLA studies a smaller policy in the same end-to-end family [22, 24]. Hybrid systems separate more of the decision process: SayCan combines language-model scores with learned affordances over a predefined skill set [6]. OWL-TAMP uses a vision–language model to propose a partial plan skeleton and discrete and continuous constraints, after which a task-and-motion planner produces a complete plan that satisfies them [9]. A recent structured long-horizon manipulation study also compares VLA policies with neuro-symbolic sequencing; its result is specific to that task setting rather than a universal ranking of the two families [3]. Table 4.1 summarises the resulting design comparison.

Table 4.1: Representative architectures compared by the roles assigned to learning, planning and command execution.

Approach	Learned output	Planning role	Execution path	Decision for Room 315
End-to-end VLA policy	Robot-action tokens or trajectories	Action selection inside the learned policy; no separate symbolic planner	Decoded action passed to the robot control stack	Not selected: the training corpus labels scene state rather than action trajectories, and the rail rules were already explicit.
VLM/LLM with skills or a planner	Ranked skills, subgoals or planning constraints	Skill selector or task-and-motion planner	Selected skill or downstream controller	Related design pattern, but not adopted as-is: open-world constraint generation was unnecessary for the fixed topology.
Neuro-symbolic planning with learned control	Symbolic abstractions and low-level continuous-control policies learned from demonstrations	A PDDL planner determines the high-level sequence	Learned controller executes each planned operator	Closest architectural family; adapted because Room 315 learns task and scene inputs but retains an explicit controller.
Implemented Room 315 system	Schema-constrained task fields and paired-camera state proposal; neither contains a plan or action	PlanSys2/POPF for normal planning; the stepwise executive handles one route-normalisation prerequisite	Supervisor publishes a typed command; the rail controller applies it	Implemented: matches the state-labelled corpus, fixed topology and simulation objectives.

For the rest of the report, VLA refers to the Phase 2 research direction and to learned-action systems such as RT-2. The Room 315 implementation is described as a neuro-symbolic language-and-vision interface.

4.6 Visual-state model

The perception baseline uses ResNet-18 as a compact backbone [5]. The fixed-camera, fixed-fleet formulation produces the compact state required by the Room 315 demonstrator and is specific to this room and fleet. Its inputs, outputs, presence handling and training process are specified in chapter 9. The simulated command and state checks performed by the software supervisor are defined in chapter 8; their engineering limits are discussed in chapter 12.

Chapter 5

Building the MFJA Digital Twin

5.1 Reconstruction strategy

The work began by creating a minimal model of the MFJA third floor. The first objective was to establish a recognisable and geometrically coherent environment before adding behaviour. The third-floor mesh was imported and reoriented, the ground plane was enlarged, and the Room 315 geometry was completed with doors and an interior window. Furniture, protective guards, robot tables and laboratory assets were then added.

A usable digital twin requires each asset to have consistent units, origin, pose, visual material and collision representation. CAD sources frequently place the origin at a manufacturing datum that is inconvenient for simulation, while Stereolithography (STL) files contain no unit metadata. Explicit scale and orientation checks were therefore used to align the visual and collision geometry with the intended room coordinates. The integration process was incremental:

1. load one asset in an isolated scene;
2. verify scale and orientation against known room dimensions;
3. establish a stable model frame;
4. separate visual and collision geometry where necessary;
5. place the asset in the full floor;
6. add a static test for paths and SDF structure.

The KUKA model followed the same process, with a targeted and documented collision-mesh configuration that preserved the required collision behaviour.

5.2 World decomposition

The final description package provides three principal world configurations:

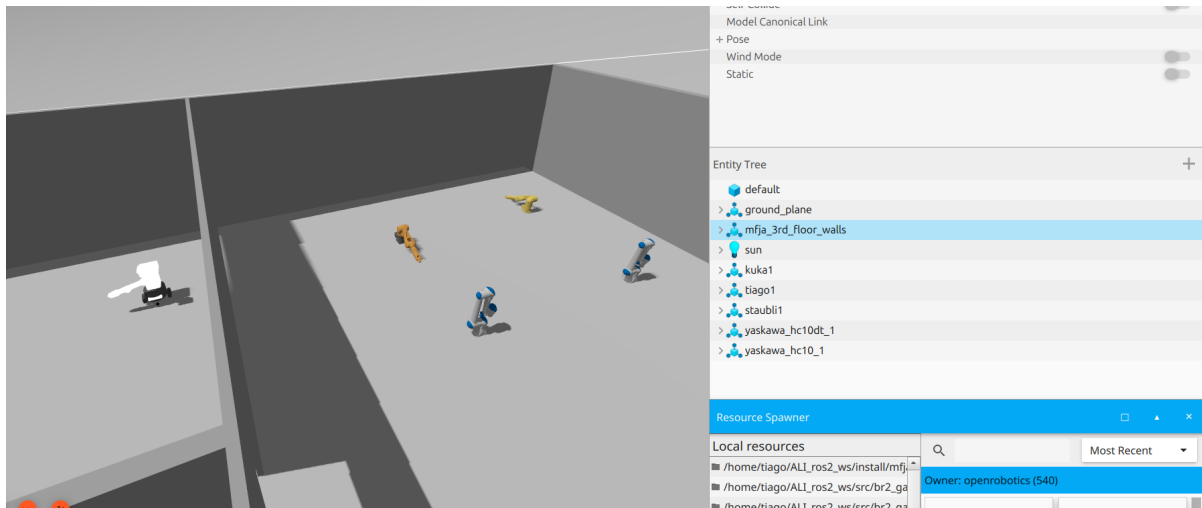
- `mfja_3rd_floor.world` for the shared third-floor environment;
- `room_315_only.world` for fast, focused rail, perception and language-integration work;
- isolated industrial-robot worlds assembled by launch for controller and geometry checks.

The internal Gazebo world name forms part of the interface because the pose-update, entity-creation and removal services include it in their service names. The bring-up package passes both the selected world and its internal name explicitly, keeping the shuttle-controller services aligned with the active world. The runbook also includes checks of the available services. The reconstruction progression is shown in figure 5.1.

5.3 Repository refactoring

As the number of assets and launch components grew, the original flat layout was refactored into a meta-repository on 25 March. The root ceased to be a ROS package, and four packages were assigned separate responsibilities:

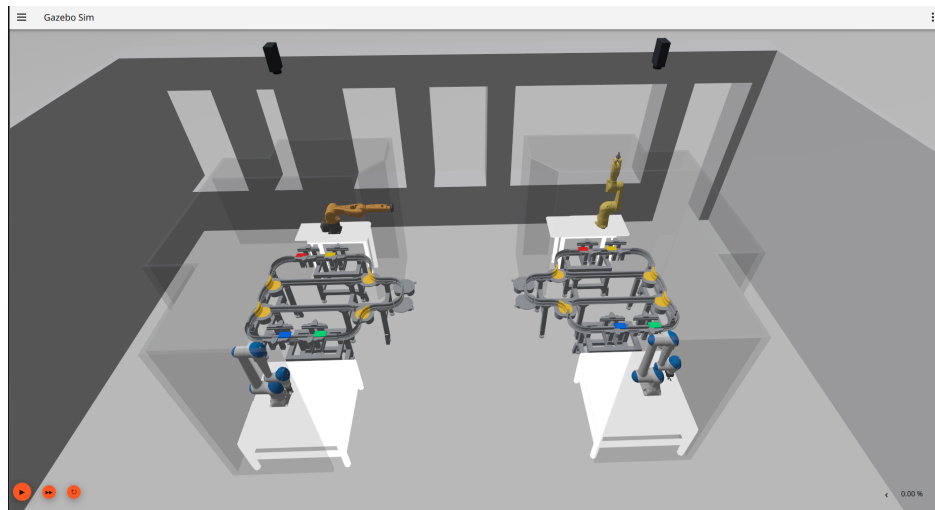
mfja_3rd_floor_description SDF, URDF, worlds, models, meshes and static description tests;
mfja_3rd_floor_bringup user-facing launch entry points and shared floor/Room 315 orchestration;
mfja_rail_interfaces custom messages and the shuttle-add service;



(a) Initial minimal floor model.



(b) Reconstructed third-floor context.



(c) Room 315 cells and robots.

Figure 5.1: Reconstruction from the initial minimal scene to the complete building context and the detailed Room 315 model. The cell provides the common workspace for transport, perception, planning and supervised execution experiments.

mfja_robot_control_config robot configuration, rail controllers, planning, perception, learning tools, launches and tests.

This organisation gives the dependencies a clear direction. Description assets remain separate from planning code, and the interface package is independent of the controller implementation. The bring-up package composes the system without containing its control algorithms, while the control/configuration package depends on the description and interface packages. This separation also supports selective `colcon` builds. The ownership and composition boundaries are shown in figure 5.2.

ROS 2 package architecture

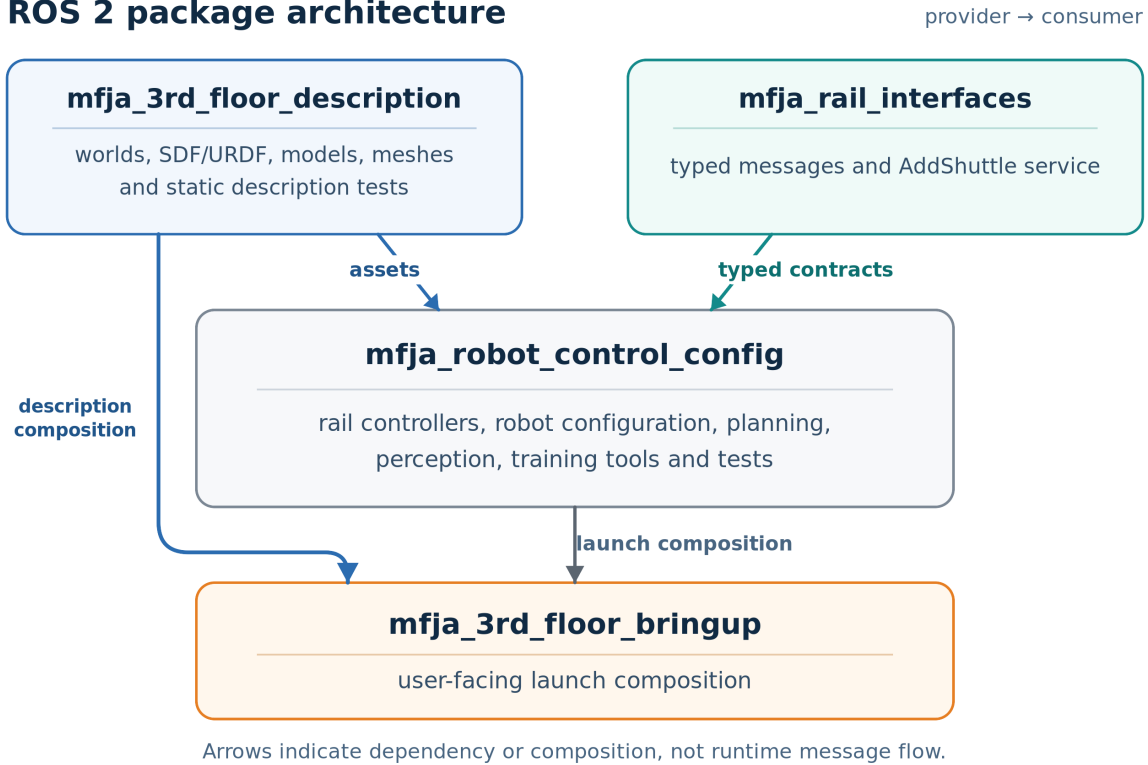


Figure 5.2: ROS 2 package ownership and composition. Description assets and typed rail contracts feed the control/configuration package; the bring-up package provides user-facing composition. Arrows represent dependency or launch composition, not runtime ROS message flow.

5.4 Model integration

5.4.1 Industrial robots

The integrated fixed-base robots are a KUKA KR6 R900 sixx, a Stäubli TX2-60L, a Yaskawa HC10 and a Yaskawa HC10DT. Each model includes a visual and collision tree, support geometry and a controller-facing joint list. The integration preserves vendor-specific joint structure but exposes a common higher-level command tool.

5.4.2 TIAGo

TIAGo differs from the industrial arms because it has a mobile base, torso, arm and head; a base-only variant is also available for focused tests. The corresponding embodiment-specific command and feedback surfaces are described in chapter 6.

5.4.3 Room and device assets

In addition to the room shell, Room 315 contains separate models for the static left and right cell frames, rail paths, switches, shuttles, carried payloads, cameras and visual markers. The markers support inspection and can be hidden for clean camera capture. The payload model distinguishes the two runtime states used by the task: a shuttle is either *loaded* or *empty*.

5.5 Launch architecture

The bring-up package exposes three stable entry points: `full_floor.launch.py`, `room_315_only.launch.py` and `single_industrial_robot.launch.py`. Shared Room 315 arguments are declared once in `room_315_floor_common.py`. This avoids configuration drift in which the full-floor and room-only launch files gradually develop different parameter names or defaults.

Launch choices include graphical user interface (GUI) or headless mode, pause state, selected robots, kinematic shuttles, Room 315 overhead cameras, visual markers, active shuttle identities, starting slots and initial loaded-shuttle selection. The default configuration leaves the scene inactive or readily inspectable, while computationally expensive or research-specific nodes are enabled explicitly. The launch layer also starts the ROS–Gazebo bridges required for entity creation, removal and pose updates.

Listing 5.1: Focused Room 315 launch used as the basic transport test.

```
1 ros2 launch mfja_3rd_floor_bringup room_315_only.launch.py \
2   robots:=none \
3   start_paused:=false \
4   gui:=true \
5   enable_room315_kinematic_shuttles:=true
```

5.6 Operational profiles

The launch layer combines the same installed assets into several repeatable experimental profiles. An operator can isolate one integration boundary, control the computational load and then scale to the shared environment without changing package ownership or public namespaces. The available profiles are listed in appendix B, and their verification status is presented in chapter 11.

5.7 Build and environment reproducibility

The baseline consists of Ubuntu 24.04, ROS 2 Jazzy and Gazebo Harmonic in a standard `colcon` workspace. The package install rules cover the scripts, configuration files, launch files, models and interfaces required by the runtime. The recorded campaigns and the full-floor smoke were launched through the installed ROS 2 package surfaces. The environment and build procedure are specified in appendix A.

5.8 Outcome and model fidelity

This phase produced usable simulation models of the full floor and Room 315, selectable robots and a package structure that supports the subsequent transport, perception, language interpretation and planning work. The recorded evaluations include one all-robot full-floor cold start and the Room 315 integrated campaign; later chapters report transport, perception and result checking separately. The transport model and its kinematic assumptions are detailed in chapter 7; the next evaluation steps are discussed in chapter 12.

Chapter 6

Unified Multi-Robot Access and Control

6.1 What “unified” means

In this project, a *unified* access layer is defined by four concrete properties:

1. common YAML configuration declares selectable robot instances;
2. a common launch path spawns the selected instances and their bridges;
3. a separate common command helper validates a named robot and its joint vector;
4. instance-derived namespaces isolate commands and feedback.

The common layer preserves the differences that matter to operation. Fixed-base arms use joint trajectories, while TIAGo additionally exposes base velocity and a linear torso joint. Joint limits, counts, names and units remain robot-specific. The common joint-command helper validates the selector, vector and units for `/joint_trajectory`; interfaces specific to a particular robot structure, such as `/cmd_vel`, remain separate.

6.2 Multi-robot operation in one Gazebo process

The first phase established per-instance command interfaces for heterogeneous robots within one Gazebo process. Repeatable operation depends on a consistent identity chain. Each selected YAML entry defines the instance name, model, spawn pose and enabled state. The shared launch rejects duplicate names and derives the namespace and bridge rules from the same identity. Command-helper profiles are validated separately because joint order and units depend on the robot structure. As a result, selection, spawning, commands and feedback remain associated with the selected instance by construction. The retained live smoke covers the all-robot profile; automated suites reported in chapter 11 cover selector rules for the other supported forms.

6.3 Configuration-driven robot registry

Robot instances are selected from YAML entries containing the instance name, model, spawn pose and enabled flag. The full floor and Room 315 can use different files while sharing the same loader. The launch layer derives SDF/URDF paths, ROS namespace and bridge topics from those fields. Users can select a full name, derived short alias, numeric order, comma-separated set, `all` or `none`; the principal surfaces are summarised in table 6.1.

The operator helper uses a separate validated Python profile table for aliases, joint ordering, command topics and angular or linear unit semantics. Tests cover both configuration boundaries, although the YAML entries and helper profiles are not generated from a common source.

The YAML-driven launch eliminated duplicated launch scripts for every robot combination. Tests check unique instance names and asset resolution; separate helper tests check profile selection, joint counts, units and command topics.

6.4 Dynamic bridge generation

Gazebo and ROS 2 use different transports. The launch process derives bridge configuration for each selected robot and starts `ros_gz_bridge` instances. Joint commands travel from ROS 2 to Gazebo; joint state, odometry and transform information travel back where required. The design follows

Table 6.1: Principal robot selectors and command surfaces.

Selector	Instance	Type	Primary command
kuka	kuka1	KUKA KR6	/kuka1/ joint_trajectory
staubli	staubli1	Stäubli TX2	/staubli1/ joint_trajectory
hc10	yaskawa_hc10_1	Yaskawa HC10	/yaskawa_hc10_1/ joint_trajectory
hc10dt	yaskawa_hc10dt_1	Yaskawa HC10DT	/yaskawa_hc10dt_1/ joint_trajectory
tiago	tiago1	Mobile manipulator	/tiago1/ joint_trajectory /tiago1/cmd_vel
tiago_base	tiago_base1	Mobile base with torso	/tiago_base1/ joint_trajectory /tiago_base1/cmd_vel

the official bridge model, in which YAML connects a ROS topic, Gazebo topic, the corresponding message types and communication direction [15].

Deriving the bridges from the selected YAML entries keeps renamed namespaces and bridge configuration synchronised, while per-instance command topics preserve one-to-one addressing. The launch selection, spawned identity and bridge topic are therefore all derived from the same YAML entry, as shown in figure 6.1.

6.5 Joint-command helper

Publishing a raw `trajectory_msgs/JointTrajectory` message is cumbersome and prone to input errors. The `robot_joint_command.py` operator tool lists supported names, checks the expected number of positions, maps names to joints, accepts radians or degrees for angular joints and uses metres for linear joints. It then publishes only to the selected namespace.

Listing 6.1: Examples of the common robot command tool.

```

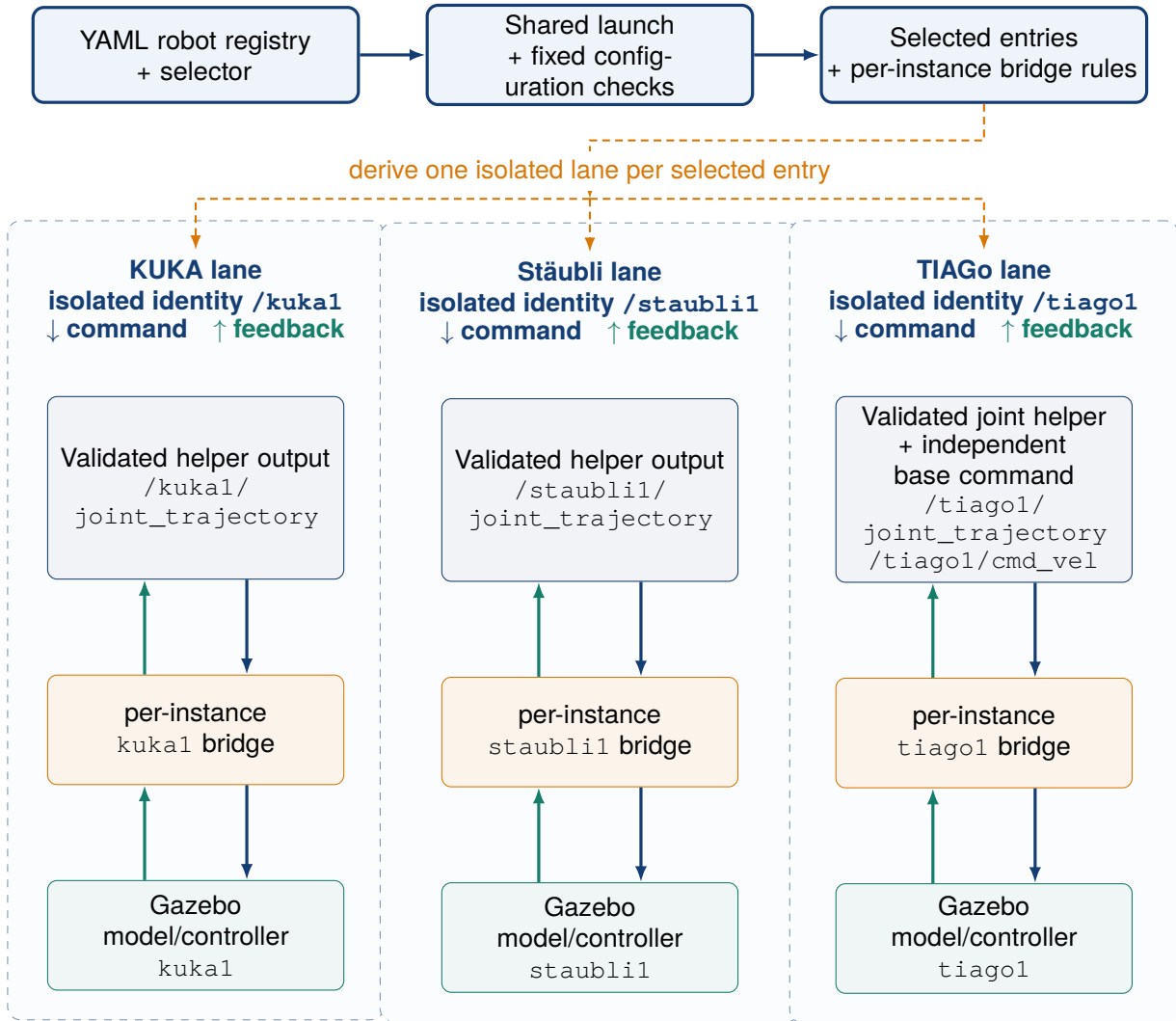
1 ros2 run mfja_robot_control_config robot_joint_command.py --list
2
3 ros2 run mfja_robot_control_config robot_joint_command.py \
4   kuka --unit deg --positions 34.38 -57.30 63.03 0 34.38 0
5
6 ros2 run mfja_robot_control_config robot_joint_command.py \
7   staubli --unit rad --positions 0.1 0.4 -0.6 0 0.5 0

```

The helper standardises operator input and creates a short trajectory command with the required joint ordering. Trajectory planning remains separate, as do collision-free arm motion, synchronisation with the transport cell and controller-specific operating constraints.

6.6 Interface portability

The public namespaces and typed state boundaries are independent of the current simulator backend. This separation makes provider substitution possible while preserving identity, units, timestamps, freshness and controller-feedback semantics. The implementation and results reported here use the Gazebo providers identified in appendix A.



Namespace isolation preserves robot identity across command surfaces, per-instance bridge, Gazebo entity and feedback.

Figure 6.1: Configuration-driven multi-robot launch, command and feedback isolation. Selected YAML entries generate per-instance bridges; the selected operator-helper profile addresses the corresponding name. There is no shared global joint-command topic.

6.7 Results

The common selector implements `none`, one exact name, comma-separated sets and `all`. In the retained all-robot cold-start smoke, the full-floor entry point spawned the six enabled registry entries. Each exposed a distinct joint command topic and a fresh namespaced state sample; both TIAGo variants also exposed their independent base-velocity topic. The smoke recorded startup and feedback without issuing motion commands. The command helper and launch-selection rules are covered by the named automated suites in chapter 11. The resulting access layer therefore brings the separate simulation models under a common, maintainable interface. The live check covers the `all` selector, while the automated tests exercise the remaining selector rules.

Chapter 7

Room 315 Kinematic Transport System

7.1 Modelling decision: dynamic or kinematic

At the beginning of the rail implementation, two motion strategies were considered:

Dynamic wheel–rail simulation Model every shuttle as a driven rigid body and let the Gazebo physics engine determine its motion from wheel or guide contacts with the rail, including the changing geometry at switches.

Kinematic rail-following simulation Represent a shuttle by its current graph segment and arc-length coordinate, compute the corresponding pose from the calibrated rail centre-line, and publish that pose to Gazebo.

The dynamic option would be appropriate for estimating traction, contact loads, wheel slip, derailment behaviour or mechanical wear. For the present objectives, however, it would add substantial cost without providing reliable additional information. Each shuttle would introduce several simultaneous wheel/guide–rail contact points, while the switch regions would continually change the active surfaces. Across the eight-shuttle fleet, the physics solver would therefore have to resolve many coupled contact and friction constraints at every simulation step. A credible model would also require validated collision geometry, masses and inertias, wheel and rail profiles, friction, compliance or damping, drive characteristics and switch mechanics. Smaller integration steps and additional solver iterations might then be needed to limit penetration, jitter or loss of contact, further increasing the computational load. Without measured mechanical parameters, this complexity would give an impression of accuracy unsupported by the available data.

The project instead had to test route selection, slot arrival, occupancy, switch and stopper logic, camera perception, planning and supervised execution. The kinematic option directly serves those questions. It provides repeatable trajectories and sensor crossings, predictable logical progression, lower computational cost for multi-shuttle scenarios and faster generation of consistent visual data. It also makes failures easier to localise: geometry determines the pose, topology determines the permitted successor, and the controller and supervisor determine whether motion is allowed.

7.2 Kinematic transport backend

The backend advances an arc-length state on a calibrated rail graph and publishes the corresponding Gazebo pose through the world `set_pose` service. This gives the initially static Room 315 rail assemblies predictable motion for repeatable scenario generation, while keeping the implementation focused on planning and supervised execution. The backend provides repeatable control of the route, commanded speed and sensor crossings within the stated kinematic fidelity boundary. The shuttle assets are declared kinematic, gravity is disabled and wheel–rail contact is not used to produce motion. The controller’s logical rail state is therefore the state reported to the rest of the system; asynchronous `set_pose` publication and rendering may lag behind that state without changing the logical progression.

7.3 From SolidWorks points to a continuous rail graph

The current kinematic model uses ordered rail centre-line points extracted from the SolidWorks geometry. The source set stores 276 points in 14 comma-separated-value (CSV) files; every row contains `index`, `x`, `y`, `z`. The conversion from CAD to runnable motion follows five explicit steps:

1. split the exterior, interior and connector curves at meaningful branch boundaries;
2. preserve point order inside one CSV per segment, so consecutive rows form a measured polyline traversed from index 0 to the last row;
3. associate each public segment with its source CSV and declare its start and end node in network YAML;
4. construct an arc-length parameterisation and a cubic Hermite interpolant between consecutive points, using neighbouring points to estimate tangents;
5. apply the calibrated rail-to-Gazebo transform before publishing the shuttle pose.

The CSV files describe *geometry*, while connectivity is defined separately. Exact floating-point equality between endpoints proved too fragile, so twelve explicit anchor nodes document the segment ends intended to belong to the same physical junction. The configuration records a 0.05 m tolerance for this CAD-to-topology correspondence. Runtime connectivity is taken from the explicit routing tables rather than inferred from coordinate proximity. The loader records the tolerance as configuration metadata rather than applying it during runtime routing.

Each public block has a length and a mapping between public labels and source curve segments. A shuttle state contains its current segment and arc length. The pose evaluator computes

$$\mathbf{p}(s) = [x(s) \quad y(s) \quad z(s)]^T, \quad \psi(s) = \text{atan2} \left(\frac{dy}{ds}, \frac{dx}{ds} \right). \quad (7.1)$$

The pose is then transformed into the Gazebo world frame.

Cubic Hermite interpolation was chosen because it allows the position and tangent to be controlled at segment boundaries. Linear interpolation creates visible yaw discontinuities, while an unconstrained global spline may leave the physical rail. Retained tests load every normalised segment with both path backends and preserve the declared topology; separate route and device tests cover successor selection and slot/sensor mappings. In figure 7.1, the black points are the complete ordered CSV input and the coloured curves are dense samples from the same Hermite evaluator used at runtime.

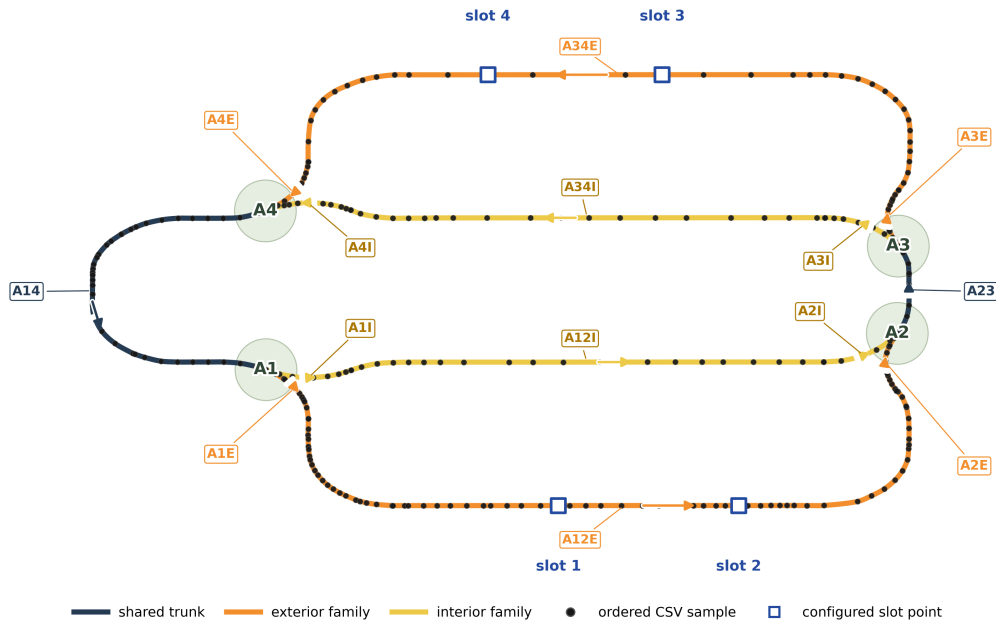
Current Room 315 right-rail source geometry: points and runtime reconstruction

Figure 7.1: Current Room 315 centre-line reconstructed from the 276 ordered CAD/CSV samples. Black dots are source samples and coloured curves are dense outputs of the runtime Hermite evaluator. The drawing uses source/CAD orientation, with the configured affine calibration mapping it into Gazebo. This figure checks reconstruction from the configured samples; an independent geometric accuracy study would require separate metrology.

7.4 Topology and switch assignments

Active routing combines the rail geometry with switch state. Each rail has four switches, each with exterior or interior state, producing $2^4 = 16$ assignments. The reference left and right network YAML files define nodes, segments, branch connections and public labels. The public segment vocabulary contains 14 blocks:

$\{A12E, A12I, A14, A1E, A1I, A23, A2E, A2I, A34E, A34I, A3E, A3I, A4E, A4I\}$.

The controller selects successor segments according to switch state. The PDDL problem generator reads the same topology, avoiding separate manually written maps for planner and controller routes. A shared topology became especially important when blocker clearance expanded beyond the normal outer loop. Table 7.1 uses only the public ROS naming domain. For the left rail, current and successor segments are converted from internal geometry identifiers to public labels, while switch identifiers already use the public physical labels. The resulting public successor contract is identical for the two mirrored rails.

Table 7.1: Public successor map shared by both Room 315 rails.

Current segment	Route switch	Switch state	Next segment
A23	A3	E	A3E
A23	A3	I	A3I
A3E	—	Any	A34E
A3I	—	Any	A34I
A34E	A4	E	A4E

Current segment	Route switch	Switch state	Next segment
A34E	A4	I	FALLING
A34I	A4	E	FALLING
A34I	A4	I	A4I
A4E	—	Any	A14
A4I	—	Any	A14
A14	A1	E	A1E
A14	A1	I	A1I
A1E	—	Any	A12E
A1I	—	Any	A12I
A12E	A2	E	A2E
A12E	A2	I	FALLING
A12I	A2	E	FALLING
A12I	A2	I	A2I
A2E	—	Any	A23
A2I	—	Any	A23

Between segments, the table—not point distance—selects the successor. “Any” marks a fixed transition, while an incompatible converging-switch state has no successor and enters FALLING. In the forward public traversal, A2 is the entry boundary of A23 and A3 selects its exterior or interior successor, as shown in figure 7.2.

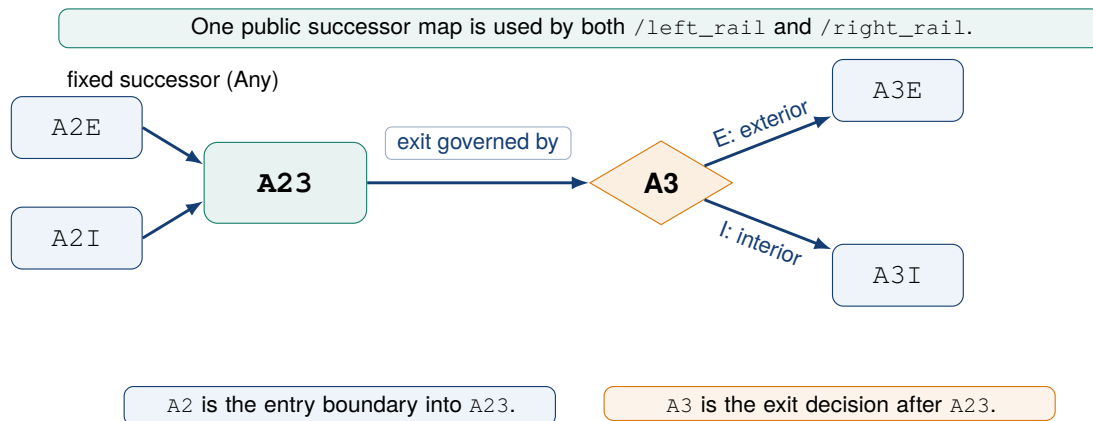


Figure 7.2: A2 is the fixed entry boundary into public segment A23. Once the shuttle is on A23, switch A3 selects the exterior or interior successor. The same public rule applies to both mirrored rails.

7.5 Devices and public naming

The physical documentation groups devices by switches, branches and indexing zones. The implementation assigns consistent public names and defines the devices in YAML. The resulting right-rail layout is shown in figure 7.3.

7.5.1 Switches

Switches A1–A4 select exterior or interior branches. Commands and feedback use separate topics. A switch update runs without globally pausing Gazebo, so other robot processes continue normally. The protected physical neighbourhood extends 0.35 m along the rail coordinate; chapter 8 defines how controller feedback authorises or vetoes a change.

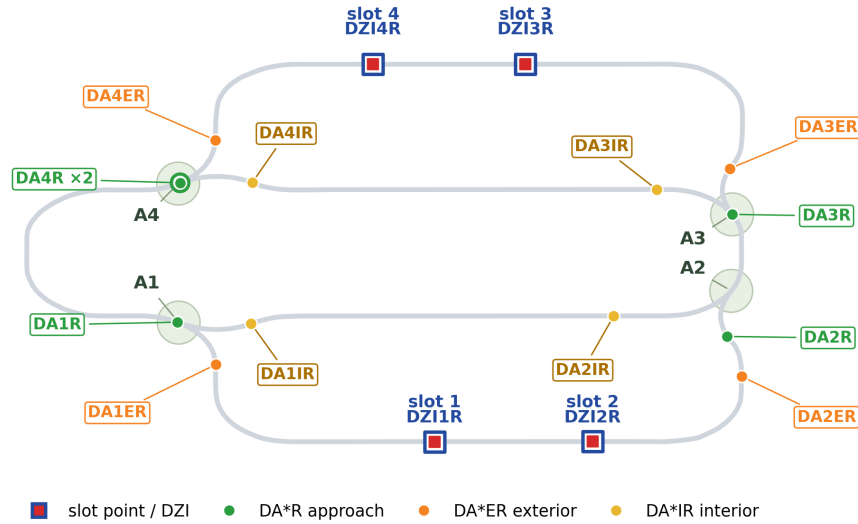
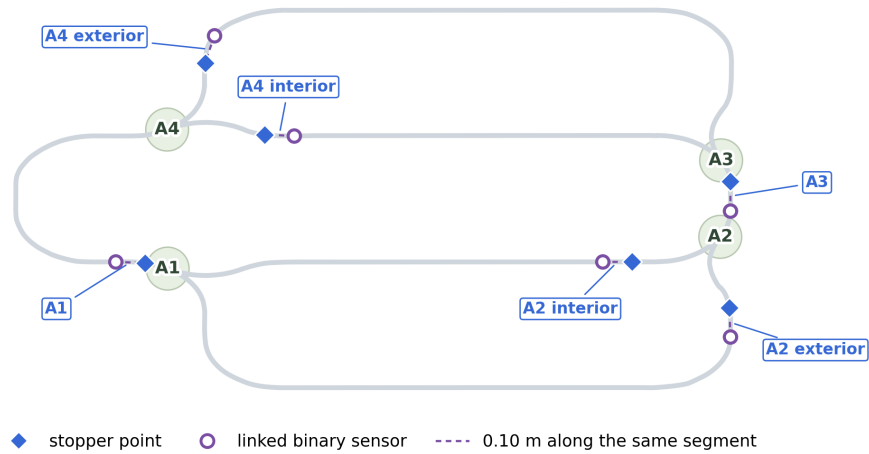
(a) Configured indexing zones and approach/branch detectors**(b) Stopper points and their derived sensors 0.10 m upstream**

Figure 7.3: Right-rail devices evaluated from the configured YAML definitions on the controller's Hermite geometry. Panel (a) shows slots and approach/branch detectors; panel (b) separates physical stopper points from their linked binary sensors 0.10 m upstream.

7.5.2 Stoppers

Stoppers hold or release a shuttle before a protected area or at a target. Their state comes directly from the rail controller. The workflow usually closes a target stopper, opens upstream stoppers, starts the shuttle and waits for the identity-bearing target sensor. A stopped shuttle can still report its configured speed setting, so the state machine uses the explicit controller mode rather than interpreting a nonzero speed field as motion.

7.5.3 Binary sensors

Sensors publish continuous binary occupancy readings, not continuous distance. Slot sensors such as DZI1R/DZI2R identify arrivals; approach and branch detectors locate a shuttle near a switch or on an inner/outer route. Continuous pose remains available inside the simulator controller for safety veto and evaluation, but it is not exposed as a learned model input.

7.6 Multi-shuttle identity and lifecycle

The current fleet supports four identities on each mirrored rail. Stable short identities L1–L4 and R1–R4 map to Gazebo entity names such as `room315_right_shuttle_4`. Identity is not inferred from array order. The operator-facing lifecycle supports:

- add/spawn through a typed low-level service with a name, start slot, speed and initial enabled state;
- per-entity ON, OFF, RESET and REMOVE commands;
- rail-wide ALL ON, OFF and RESET commands;
- enabled, disabled, moving, waiting and fault-related state;
- explicit payload state and debug visual identity.

Emergency stop disables all shuttle motion. Any subsequent execution requires an explicit operational decision.

The add service manages simulator entities. Entities created through it are registered for supervised task execution only if they match the reconciled L1–L4 and R1–R4 identities used by the language interface. This keeps the planner’s identity domain independent from ad hoc commissioning operations.

The identity registry reconciles the maximum fleet, configured identities, world entities and default spawn information. Registration proceeds only when these sources agree, ensuring that visual labels and planner objects refer to the same shuttle. Figure 7.4 shows the complete fleet in the two camera views used by the visual pipeline.

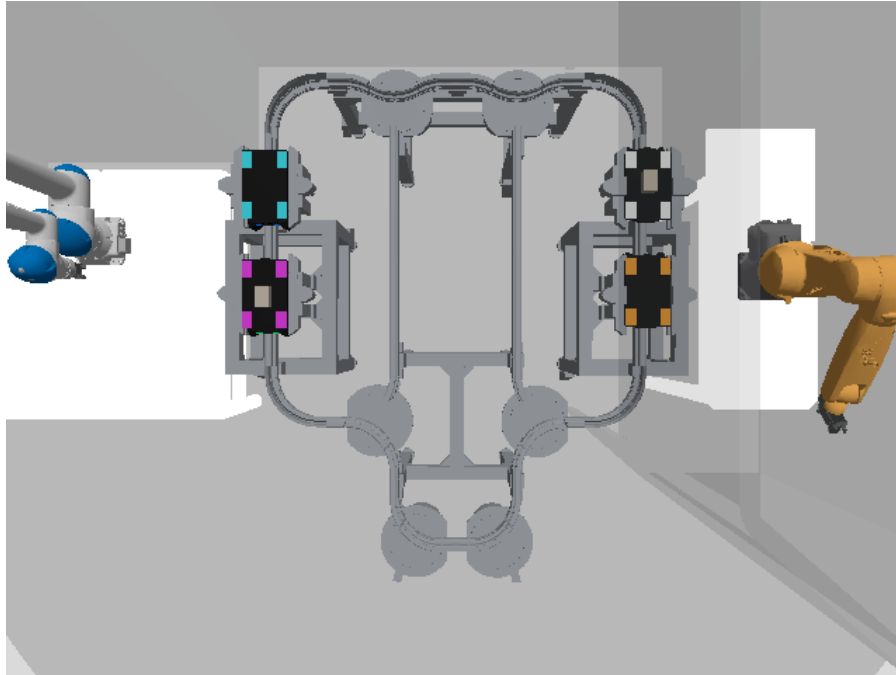
7.7 Motion, slots and occupancy

At every update step, an enabled shuttle advances by

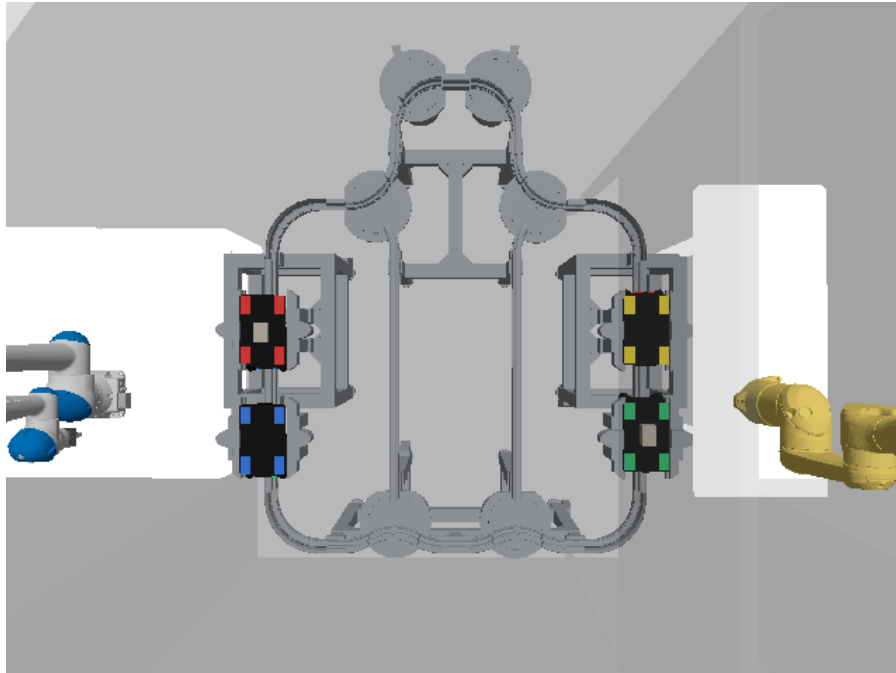
$$s_{k+1} = s_k + v \Delta t, \quad (7.2)$$

The update in equation (7.2) is subject to the active route, target, stopper and headway constraints. When the shuttle crosses a segment end, the topology selects the next segment and the residual distance is preserved. The controller update loop and configured Gazebo `set_pose` rate limit both default to 30 Hz. Device logic is evaluated on the controller’s kinematic update independently of render timing; asynchronous `set_pose` requests may reduce the achieved visual update rate.

Slots are calibrated ratios on exterior segments. Arrival requires more than proximity to the target coordinate: the low-level controller uses a target slot, the appropriate stopper, a slot detector and an explicit OFF effect. The planner maintains symbolic occupancy, while the executive verifies the observed identity at the target before declaring completion.



(a) Left cell: L1 cyan, L2 magenta, L3 orange and L4 white. The centre payload blocks are visible on loaded L2 and L4.



(b) Right cell: R1 red, R2 blue, R3 green and R4 yellow. The centre payload blocks are visible on loaded R1 and R3.

Figure 7.4: Synchronous overhead Gazebo views of the complete eight-shuttle configuration. Identity colours remain visible while device markers are hidden, making presence and payload appearance readable on both mirrored rails.

7.8 Operator-facing observation and maintenance

The runbook exposes the typed low-level controller API for commissioning and diagnosis. Direct publication can be used to isolate and test a device, while operational tasks continue to pass through the translation and supervision stages. The operator-facing interfaces are listed in appendix B.

The installed device-position tool supports maintenance of the device geometry defined in YAML. It projects a supplied Gazebo XYZ point onto the closest rail segment, reports the resulting arc length, `s_ratio`, projected point and residual distance, and defaults to a dry run. A guarded write can update slots, ordinary position sensors or stopper points; points that exceed the configured distance threshold are rejected unless explicitly overridden. Multi-point stoppers can be addressed by current segment or list index, while their linked detector positions remain derived from the stopper configuration. This tool operates offline: changes take effect only after a relaunch and must then be checked through markers and binary feedback. These maintenance and low-level interfaces remain separate from the closed-loop task interface; their commands and topics are specified in appendix B.

7.9 Headway and collision constraints

Multiple shuttles introduce a resource-allocation requirement. The controller enforces separation on the same segment; higher-level reservation ownership and authorisation are defined in chapter 8. Within the runtime, a *blocker* is another shuttle occupying the requested route, destination slot or required holding capacity. Independent objects placed on the rail are outside the current planning and execution model.

7.10 Interior staging and dense blocker cases

Both public interior branches, `A12I` and `A34I`, can serve as capacity-limited holding areas. The planner enumerates admissible holding centres from the calibrated branch length, preserves a 0.35 m end margin and the configured centre-to-centre separation, and prefers the farthest reachable free centre. With the current geometry, each branch admits at most two safely separated shuttles; the positions are calculated rather than fixed. A later `ON` command invalidates the corresponding stop certificate.

These calculated centres impose physical capacity constraints on later relocation. The closed-loop selection, re-observation and recovery policy is described in chapter 10.

7.11 Validation of the transport model

Static and behavioural checks cover calibrated geometry, topology, devices, identity reconciliation, separation and interior capacity. The corresponding results are presented in chapter 11. The physical constraints are enforced through the typed commands and supervisory mechanisms described in chapter 8.

Chapter 8

Typed Interfaces, Contracts and Safety Supervision

8.1 A typed command and state interface

The rail stack uses a dedicated typed interface package with separate command and state topics. Each message specifies its role, required fields and addressed device. This makes multi-shuttle coordination predictable and keeps requests distinct from reported state.

ROS 2 interfaces are defined using `.msg`, `.srv` and `.action` files [12]. The project defines typed messages for named state, switch command/state, stopper command/state, shuttle command/state, sensor feedback/readings and visual state, plus an `AddShuttle` service. The interface package remains independent from Gazebo-specific implementation details.

8.2 Command/state separation

The interface distinguishes three message roles:

- a *command* says what a caller requests;
- a *state* says what the responsible controller reports;
- a *result* records the operation status, the readings used to establish it and a reason where applicable.

Command publication and effect observation are separate events. The executive checks controller or supervisor state for the requested effect. Exact slot arrival requires new DZI and controller reports, while switch and stopper feedback is subject to the freshness conditions of the corresponding command-result check.

8.3 Sparse partial-update semantics

Live switch and stopper commands are sparse named updates. For example, a checked command envelope containing only `switches: {A3: INTERIOR}` requests a change to A3 and says nothing about A1, A2 or A4. An omitted device therefore means “preserve its current state”; no default value is applied to it.

The supervisor checks every named update against the current safety state. After those checks pass, it publishes a typed ROS 2 `SwitchCommand` or `StopperCommand`. Their `switches` and `stoppers` fields are arrays of `NamedState` entries, and the controller applies only the entries present in the corresponding array. Shuttle motion uses its own typed `ShuttleCommand` message. The controllers accept actuation requests through these typed command messages. The translator’s auxiliary `event_action` record and the visual model’s training-only `target_mask` serve other parts of the workflow.

Figure 8.1 shows the two rules at this boundary: execution must be confirmed through controller feedback, and only explicitly named devices may change.

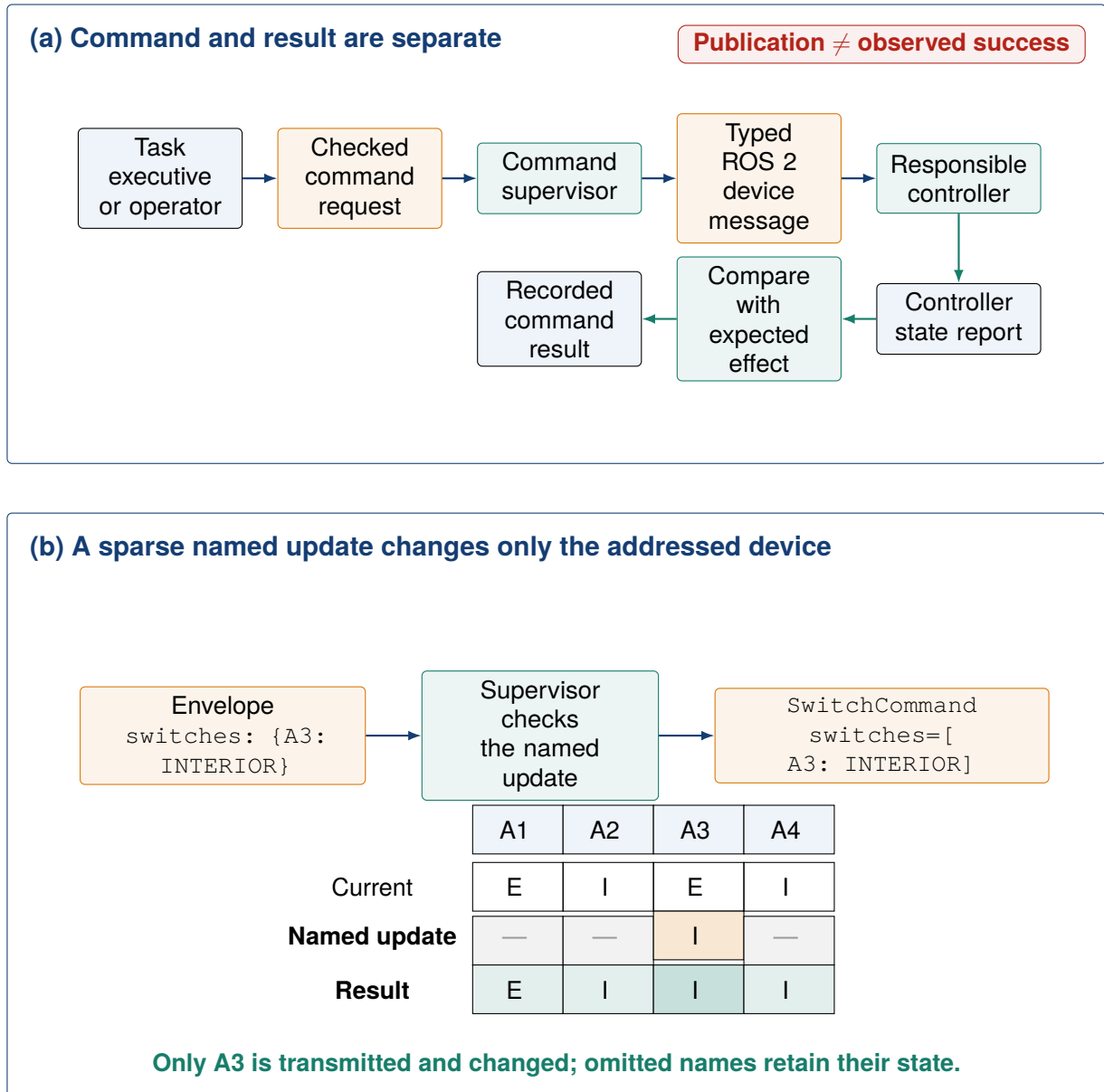


Figure 8.1: Live command semantics. The supervisor checks a sparse command dictionary before publishing typed ROS 2 device messages. Controller feedback establishes the observed result, and omitted device names retain their previous state.

8.4 Versioned contracts

Operational and task data are represented by versioned Python contracts. Planning and execution use typed records together with checked JSON command dictionaries. Stable identifiers, timestamps and producer/source information are included where required. At the execution boundary, the supervisor converts each authorised request into typed ROS 2 messages. The principal contracts and their permitted producers are summarised in table 8.1.

Table 8.1: Principal closed-loop contracts and their producers.

Contract	Meaning	Allowed producer
ObservedFact	One fact with value, confidence and known/unknown/stale/conflicting status	Visual model or rule-based provider, identified by source
ObservedState	Versioned state envelope containing visual inputs and fused facts with freshness/status metadata	State-fusion layer
SemanticParseEnvelope	Strict semantic proposal from user text	Local language model; the schema contains semantic fields and excludes PDDL, plan and command fields
TaskGoalDraft	Incomplete task proposal after field precedence and fusion	Explicit parser and field-fusion resolver
TaskGoal	Immutable validated task constraints, optionally grounded to one current instance	Domain validator; before planning, the grounding gateway may derive an instance from current visual facts
PddlPlanStep	One parsed symbolic action selected from a returned plan	PlanSys2 planner boundary; the executive alone may construct a narrowly defined route-normalisation prerequisite when required
TranslatedPlanStep	Parsed action, live sparse named command and auxiliary event-action record	Plan translator
Checked command dictionary	One physical-command request, including only the named device updates to apply	Executive or macro dispatcher; published only after supervisor authorisation
SwitchCommand/ StopperCommand/ ShuttleCommand	Typed ROS 2 execution messages; device messages carry sparse <code>NamedState[]</code> updates	Safety supervisor after current-state validation
PostconditionCheck and closed-loop records	Recorded effect status, reason and per-step/task outcome	Closed-loop executive using supervisor, controller and observation reports

The source field determines how a fact may be used. Simulator pose may provide an oracle label or a safety veto, but it is excluded from the visual model input. A language proposal passes through fusion, domain validation and operator confirmation before it becomes a task. The permitted producers in table 8.1 define these boundaries, while figure 10.1 shows the complete runtime sequence.

8.5 Observed-state semantics

Every critical fact has four possible statuses: *known*, *unknown*, *stale* and *conflicting*. This is more expressive than a Boolean value plus confidence. Unknown means the value was not observed;

stale means it was once known but is too old; conflicting means authorised sources disagree. The system preserves these statuses instead of silently coercing them to false. Figure 8.2 shows which fact statuses may authorise planner predicates and actuation.

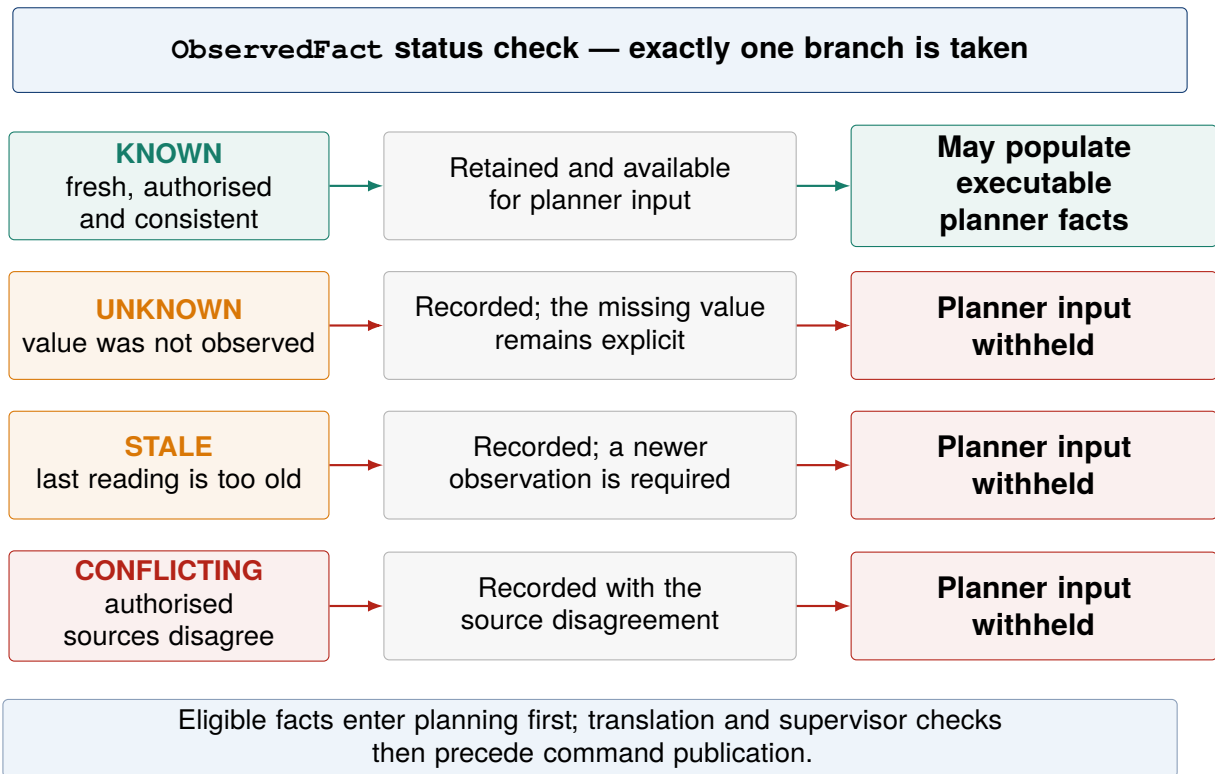


Figure 8.2: Status handling before an observed fact enters planner state.

`ObservedState` separates:

visual_model_inputs the perception facts and image-derived representation intended for the learned boundary;

fused_planner_state fused facts with their status and freshness metadata after source reconciliation.

The live task gateway retains the latest visual observation together with supervisor and device reports. The provider interface supplies one current snapshot to the executive. Live operation keeps oracle and visual providers distinct, and test-only providers remain isolated from the live path.

8.6 Supervisor responsibilities

The Room 315 execution supervisor, implemented in `room_315_vla_supervisor.py`, provides the final software authorisation before a command is published. The filename retains the early project terminology; the component is a rule-based command supervisor, not a learned VLA policy. It subscribes to command dictionaries encoded as JavaScript Object Notation (JSON), validates their sparse named updates, observes the current rail state and publishes typed device or shuttle commands only after the checks pass. The kinematic rail controller consumes these messages and applies the resulting simulated state transition.

The supervisor gates include:

- emergency-stop state and global `stop_all`;
- command structure, identity and rail-side consistency;
- fresh and non-conflicting state;
- occupied target and block reservation;
- same-segment and block headway;

- switch change within the 0.35 m protected distance;
- stopper and route-clearance state;
- newly observed occupancy by another shuttle and sensor dropout;
- timeout, unknown localisation and impossible/falling pose;
- limited retry and lexicographic reservation-conflict tie-breaking.

Safety metrics and recent decisions are stored for diagnostics. For every published command, the supervisor records the reason for authorisation. A rejected command leaves the requested device unchanged and records the reason for rejection. If an execution exception occurs, or the system shuts down during an active goal, the gateway sends `stop_all`.

8.7 Reservations and deadlock

A block reservation associates a route resource with one shuttle and task. Reservations prevent two primitives from entering the same protected block. A separate headway check prevents physical overlap on the current segment even when the destination block is free.

When crossed reservations form a detected deadlock, a lexicographic identity tie-break grants priority to one identity. The other participants remain safely stopped before observing and planning again. The implemented detector handles these crossed reservations; temporal non-progress and larger wait-for cycles would require a separate detector. Any subsequent recovery motion still requires a planned, reachable destination, available capacity and an effect check.

8.8 Source of switch feedback

Switch safety relies on a clear separation between data sources. The planner uses current visual locations, in keeping with the visual-state objective. The switch supervisor checks continuous controller coordinates before allowing a change. This check is independent of the learned observation.

The controller coordinate is mapped to the canonical rail graph and the distance to the controlled switch node is measured along the rail, not as a straight Euclidean distance through space. A switch operation is authorised only from current, finite and graph-mappable state; all other cases remain inhibited under the fail-closed policy.

8.9 Checking command effects

The runtime defines the expected effects of every translated execution command. Examples include:

- `SET_SWITCHES`: requested switch values match controller status;
- `SET_STOPPERS`: requested stopper values match controller status;
- `commanded slot motion`: the identity-bearing DZI, controller `reached_target_slot`, explicit `DISABLED` state and required confirmation frames agree;
- `STOP_NOW`: controller reports a fresh explicit `DISABLED` state;
- `observation-only inspection`: a newer visual observation is available after the request.

The executive associates the result with command, plan-step and observed-state identifiers. Motion arrival and retries following motion require newer state reports. Command-linked observation identifiers are recorded for motion effects; switch and stopper matching currently permits cached state.

8.10 Manual and emergency paths

Manual command dictionaries remain available for device debugging. Except for the dedicated emergency stop, they pass through the same structural and supervisor checks. The emergency path

has higher priority and broadcasts a stop without waiting for planner state. A stopper can therefore be tested without a language model while remaining subject to the current safety state.

Chapter 9

Developing the AI Inputs: English Understanding and Visual-State Learning

9.1 Phase 2 objective and implementation sequence

Phase 2 extended the Room 315 transport system with two AI-based inputs. Its objective was to let an operator state a task in English and use the current paired-camera observation to obtain the rail state needed to plan and complete the requested shuttle motion. The language path had to produce a task description compatible with planning, and the image path had to describe the current rail scene.

Development began with two separate questions: could an English request be turned into a valid Room 315 task, and could the paired cameras recover the scene information needed for planning? The two paths were tested separately before they were connected. The implementation sequence is summarised in table 9.1.

Table 9.1: Phase 2 development sequence.

Step	Work performed	Usable result
1	Define language and visual outputs	Semantic proposals contain task fields, while visual proposals contain scene-state values
2	Integrate a local English model and build task validation	English requests become confirmed <code>TaskGoal</code> objects
3	Generate paired-camera Room 315 data and separate the inputs from labels	A structured corpus represents shuttle presence, location and payload state
4	Partially fine-tune a paired-view ResNet-18 and select a checkpoint	Two RGB views produce a fixed-order visual-state proposal
5	Connect both branches to the runtime checks	A confirmed <code>TaskGoal</code> and a current <code>VisualStateObservation</code> form the two integration contracts

9.2 Reused components and project-specific development

The two learned components used different adaptation methods. The language model was integrated as a pretrained, quantised model without fine-tuning on Room 315 data. The visual network also reused pretrained image features, but it was subsequently trained on the project's paired-camera corpus.

9.3 Building the English task interface

9.3.1 Local model selection and configuration

The selected semantic model is `Qwen2.5-1.5B-Instruct-GGUF` from the Qwen2.5 family [23], using the `qwen2.5-1.5b-instruct-q4_k_m.gguf` quantisation. It runs locally on the CPU through `llama.cpp` [4]; the configured artefact hash is listed in appendix A. The runtime is offline and uses a 4096-token context, temperature 0, at most 192 generated tokens and seed 315. These

Table 9.2: Reused foundations and project-specific Phase 2 work.

Component	Reused foundation	Developed in this project
English understanding	Pretrained Qwen2.5-1.5B-Instruct weights and the local <code>llama.cpp</code> inference backend	Saved local configuration, constrained prompt, strict JSON schema, explicit-fact extraction, field fusion, clarification, validation and confirmation dialogue
Visual state	ImageNet-pretrained TorchVision ResNet-18	Gazebo corpus, paired-view architecture, 200-value target, masked multi-head loss, partial fine-tuning, grouped evaluation and integration of the selected checkpoint
Planning	PlanSys2 and its POPF solver	Room 315 PDDL model, fresh-problem generation and closed-loop integration (chapter 10)

settings record and control the inference conditions and keep each response short enough for the schema check.

The pretrained weights were used without project-specific language-model training or fine-tuning. A versioned prompt constrains the general instruction model to extracting a semantic proposal from an English Room 315 request. The prompt enumerates the permitted task fields, identities, rails, stations and slots. It requires one JSON object of type `SemanticParseEnvelope` and excludes PDDL, plan steps, switch updates, stopper updates and shuttle commands.

9.3.2 From model output to a confirmed task

The language path contains four project-specific stages. A rule-based extractor first records facts stated explicitly, such as *green shuttle*, *right rail* or *slot 2*. Qwen then proposes candidate intent fields such as transport, inspection, nearest or payload-qualified selection. The fusion resolver applies fixed precedence, preventing a semantic proposal from overwriting an explicit user fact or confirmed context. This produces a `TaskGoalDraft`, which is not an executable command. The dialogue manager requests any missing information, the domain validator checks rail, identity, station and slot consistency, and the operator confirms the interpretation. An immutable `TaskGoal` is published after these stages are complete.

For example, the request “Move the green shuttle to slot 2.” contains a configured colour alias. The explicit extractor resolves *green* to `room315_right_shuttle_3`; the verb and destination are also grounded directly from the text. The semantic envelope is stored with the parse record, and the explicit fields retain precedence. The confirmation therefore states R3, the right rail and slot 2 before the task is published. The complete colour-alias execution is followed in table 10.2.

The indirect request “Could you get a loaded shuttle over to the right slot 4?” falls outside the explicit transport-verb patterns. In the saved evaluation record, the semantic model supplied `goal_type=transport`, while field fusion and domain validation constrained the side, payload and destination. A separate ten-case English corpus covers explicit, indirect, ambiguous, inconsistent and cancellation forms; its evaluation is reported in section 11.3.1.

9.4 Generating the Room 315 visual corpus

9.4.1 Systematic simulated scenes

For identities declared present by the configured registry, the visual branch estimates the location and loaded state required by the planner. The dataset generator first enumerated every non-empty subset of the eight canonical identities L1–L4 and R1–R4, giving $2^8 - 1 = 255$ presence configurations. Eight geometrically different scenes were generated for each configuration by varying valid rail positions, payload states and switch arrangements, producing 2040 scenarios.

For every scenario, the orchestration layer configured the Gazebo scene, allowed it to settle and captured one synchronised image from each overhead rail camera. The resulting corpus contains 2040 paired observations, or 4080 RGB image files. Figure 9.1 shows one such model input.

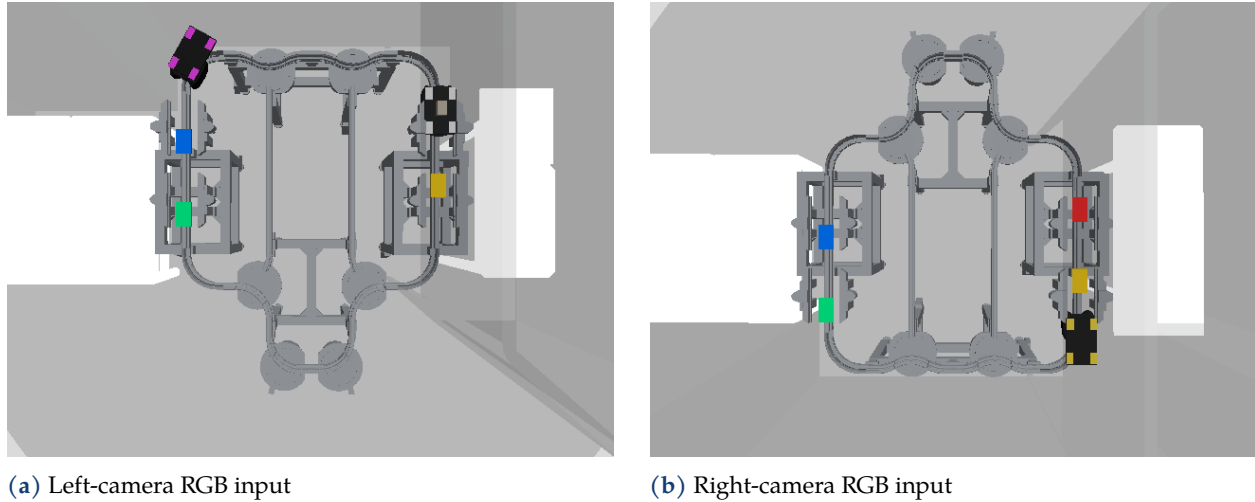


Figure 9.1: The two synchronised RGB images form the complete learned input for one scenario. Simulator state is used to construct labels in a separate file and is never appended to the image tensor.

9.4.2 Inputs, labels and grouped partitions

Gazebo supplies the known identity, calibrated rail coordinate, payload state and projected image bounding box used to create supervision. Those values are stored separately from the image references. The model features are the paired RGB overhead images; depth, controller position, point clouds and row-level oracle metadata remain label-generation data. This separation prevents simulator truth from leaking into inference.

All eight variants of one presence configuration remain in the same data partition. The resulting grouped split contains 191 configurations and 1528 scenarios for training, 32 configurations and 256 scenarios for validation, and 32 configurations and 256 scenarios for the locked Test partition. Image hashes and configuration-family membership are audited across the partitions. Checkpoint selection uses the validation partition alone; the Test partition is used once for the final evaluation in chapter 11.

9.5 Designing and training the visual-state estimator

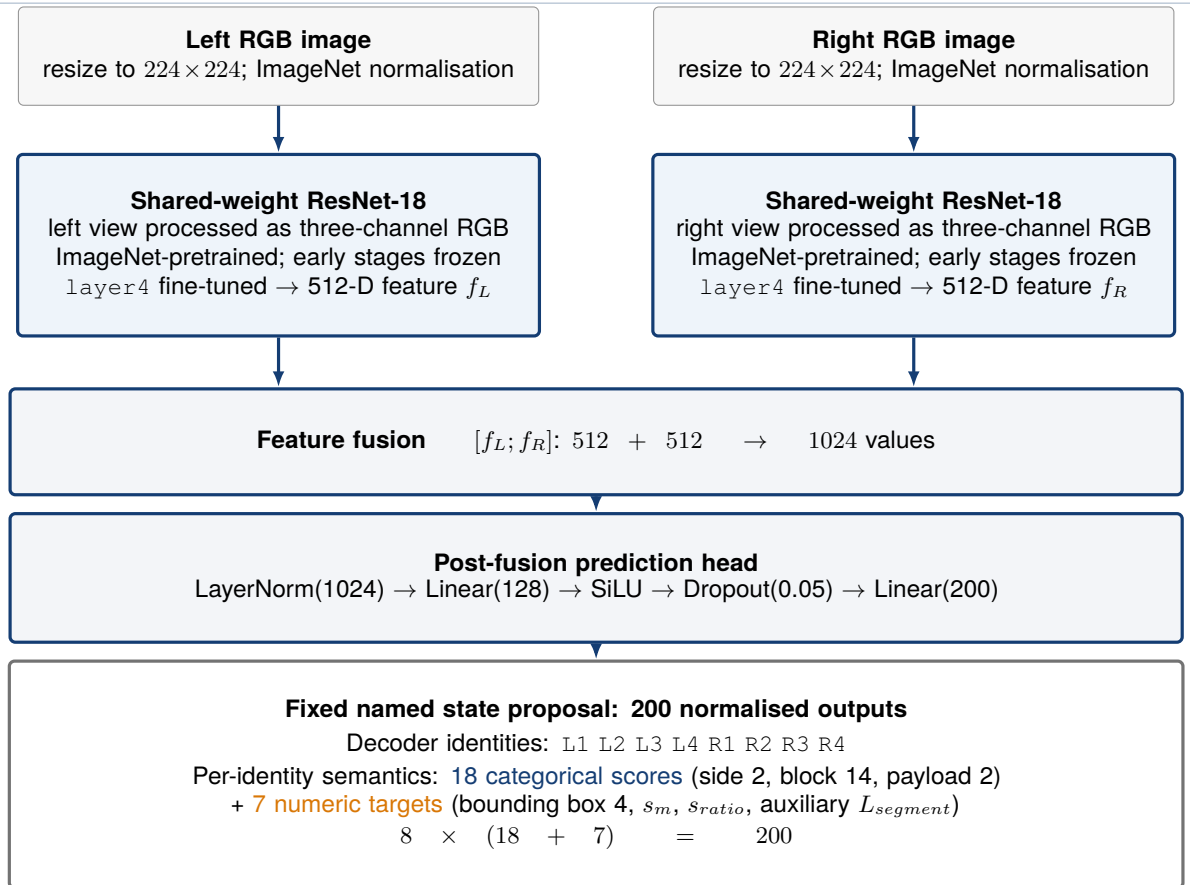
9.5.1 Fixed 200-value supervision contract

Each image pair is resized to 224×224 per view and normalised using the ImageNet channel statistics. The same ResNet-18 backbone [5] is applied to the left and right images. Their two 512-value feature vectors are concatenated, passed through a 128-unit structured head and mapped to exactly 200 output values. The fixed identity order is L1–L4 followed by R1–R4, with 25 values assigned to each identity:

$$25 = \underbrace{2}_{\text{rail-side scores}} + \underbrace{14}_{\text{block scores}} + \underbrace{2}_{\text{payload scores}} + \underbrace{4}_{\text{bounding box } (x,y,w,h)} + \underbrace{3}_{s_m, s_{ratio}, L_{segment}}.$$

The fixed output index supplies shuttle identity, while a configured registry specifies which identities exist in the launched scene. At runtime, the model estimates location and payload values for identities marked present by that registry.

A LEARNED PAIRED-VIEW STATE ESTIMATOR



B RUNTIME DECODING AND CHECKS

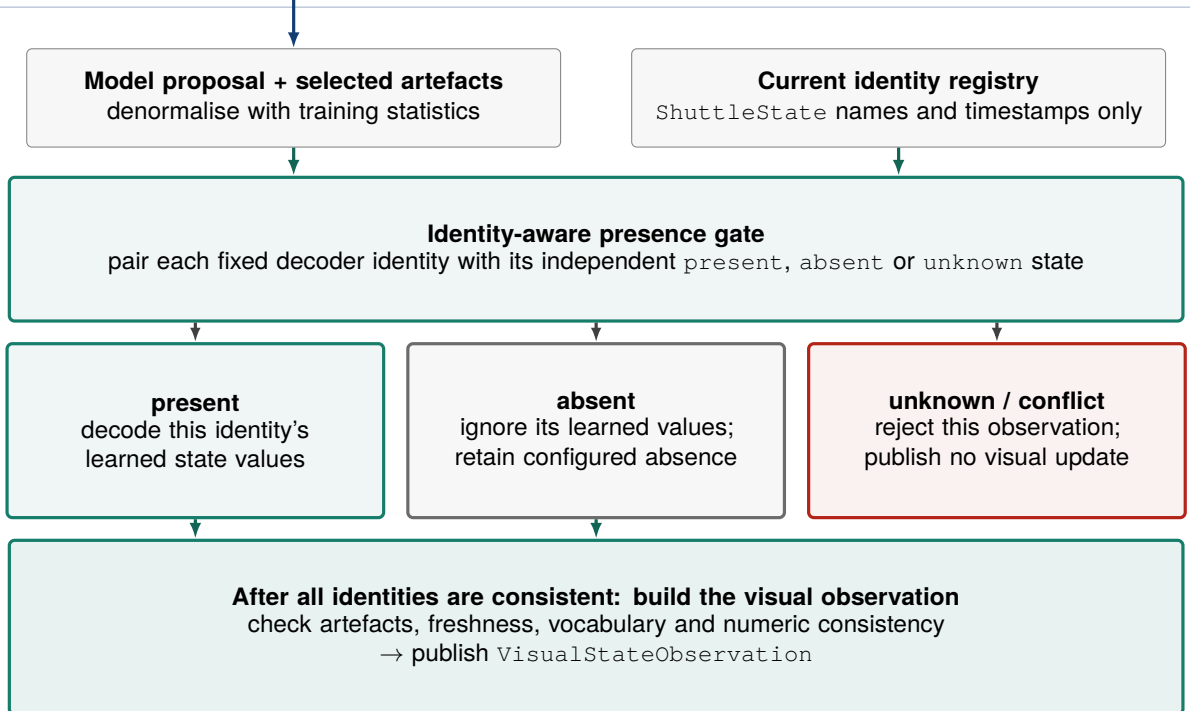


Figure 9.2: The two RGB views are encoded independently with the same ResNet-18 weights. Their 512-value features are concatenated and mapped to a fixed 200-value state proposal. A separate configured identity registry controls which named values may be decoded; whole-observation validation decides whether a VisualStateObservation is published.

The separate 200-element `target_mask` is used only to compute the loss. When an identity is absent or lacks valid visual supervision, its 25 target coordinates receive zero weight; valid present identities contribute normally. The mask is not used for image segmentation or as a runtime ROS message, and it remains outside the command path.

9.5.2 Training procedure

Training started from the official `ResNet18_Weights.IMAGENET1K_V1` checkpoint. The earlier backbone stages were frozen, while ResNet `layer4` and the custom prediction head were fine-tuned. This left 8,552,776 of 11,335,560 parameters trainable. Targets were normalised using statistics computed from valid training coordinates only.

The loss is the equally weighted mean of five masked Smooth-L1 heads: segment location, loaded state, bounding box, s_m and s_{ratio} . Equal head weighting prevents larger coordinate groups from dominating by element count, while masking removes absent identities from the loss. The exact training configuration is summarised in table 9.3.

Table 9.3: Configuration of the selected visual training run.

Item	Configured value
Input	Two RGB views, each resized bilinearly to 224×224 and ImageNet-normalised; no data augmentation
Initial weights and adaptation	ImageNet-pretrained ResNet-18; partial fine-tuning of <code>layer4</code> plus the structured head
Optimiser	AdamW, learning rate 10^{-3} , weight decay 10^{-4}
Batch and schedule	Batch size 32; maximum 15 epochs; validation-loss patience 3
Loss	Masked per-sample Smooth-L1, equal weight for five output heads
Reproducibility	CUDA training, seed 31520260730; Test partition excluded from training and model selection

All 15 epochs completed. Epoch 14 had the minimum total weighted validation loss and was therefore saved as `best.pt`. Its checkpoint, vectoriser, target statistics, preprocessing description and training configuration were stored together. At startup, the runtime checks their expected hashes and schema before the model can publish a proposal. The final locked-Test measurements are presented in chapter 11.

9.6 Integration outputs

The resulting integration contracts are a confirmed `TaskGoal` and a current `VisualStateObservation`. Chapter 10 follows how the gateway uses them for planning and shuttle motion.

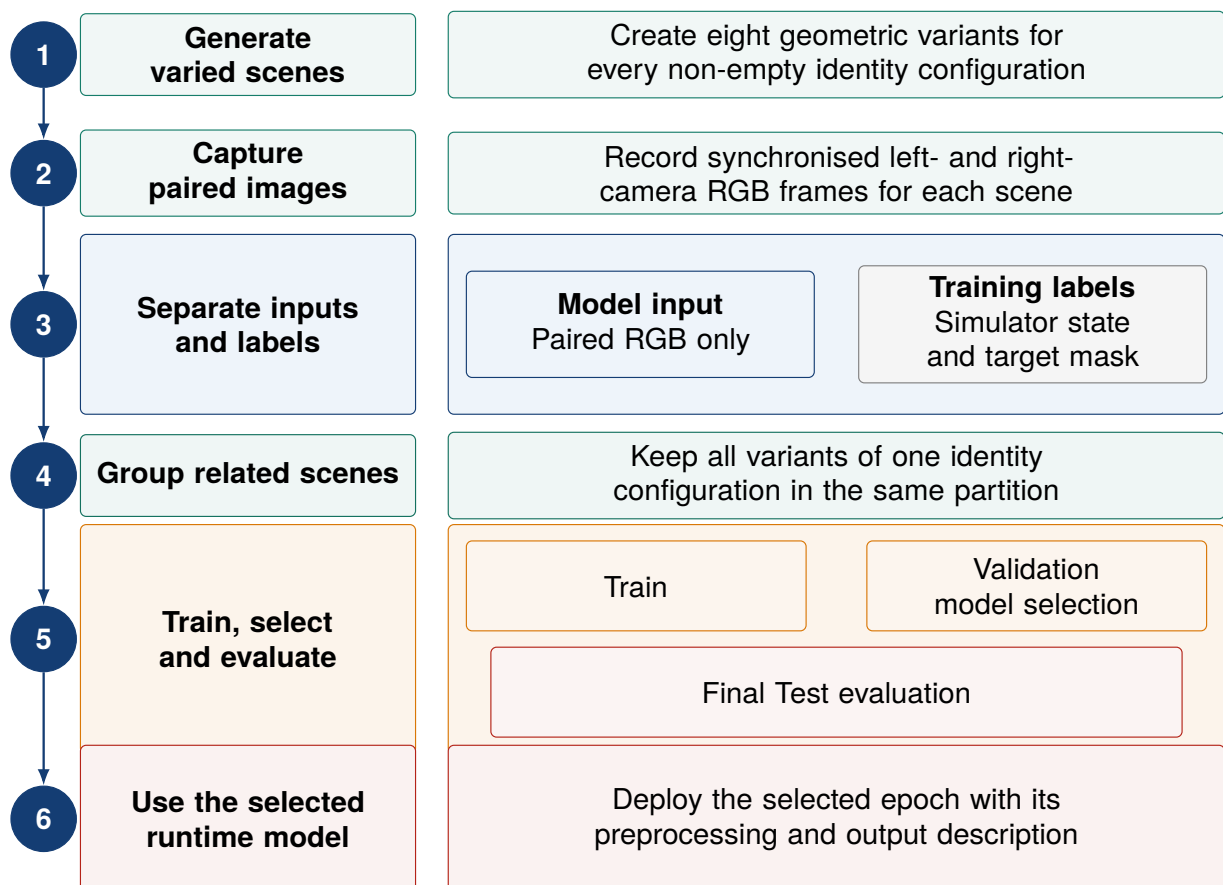


Figure 9.3: The visual-data workflow from scene generation to the model used at runtime. Paired images are kept separate from their training labels, and all variants of one identity configuration remain in the same partition.

Chapter 10

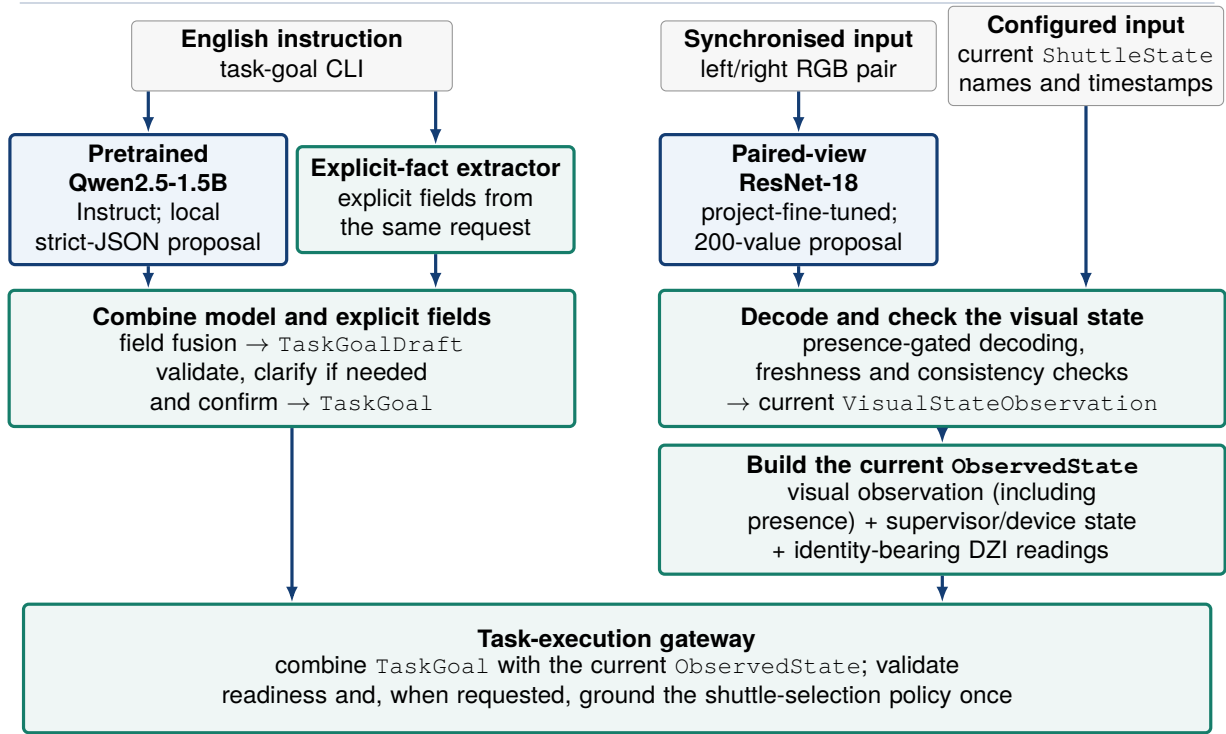
From an English Instruction to Shuttle Motion

10.1 Runtime architecture

Figure 10.1 follows the runtime from interpretation to an observed result. Qwen supplies candidate task fields and the paired-view ResNet-18 supplies scene estimates. After task confirmation and state checks, the gateway builds a PDDL problem, requests a PlanSys2/POPF plan and selects its first supported step. The supervisor publishes the corresponding typed ROS 2 command, the rail controller changes the Gazebo state, and the executive compares the resulting state with the expected effect. If the goal is not yet complete, the cycle begins again from a new observation. The interface and safety rules behind this sequence are defined in chapter 8.

The colours distinguish roles: blue boxes show learned outputs, green boxes show project logic, orange denotes symbolic planning, red marks permission to publish an actuation command, and grey denotes the Gazebo rail controller.

A INTERPRET THE REQUEST AND READ THE CURRENT STATE



B PLAN, EXECUTE ONE STEP, AND CHECK THE RESULT

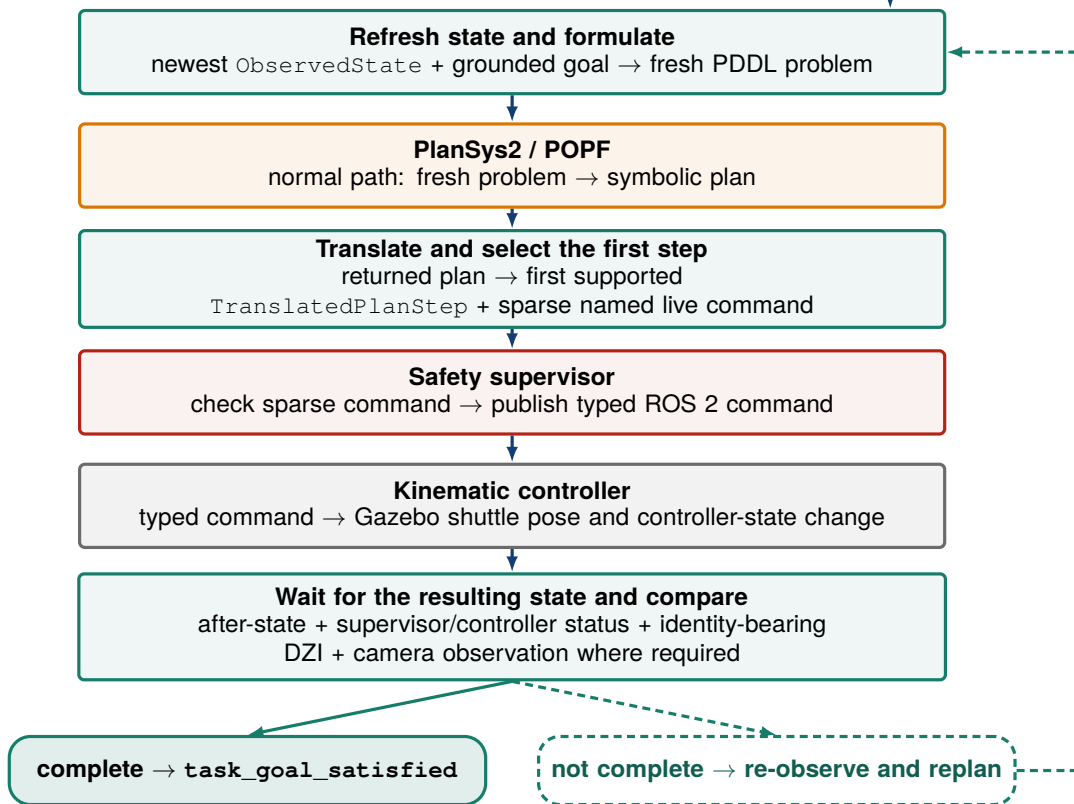


Figure 10.1: A confirmed task and current camera state feed one symbolic step. The dashed path returns to observation and planning while the goal remains incomplete.

10.2 Step 1—interpret and confirm the English task

The operator enters an English instruction through the interactive task-goal CLI. The parser pipeline runs the explicit-fact extractor and the local Qwen semantic model described in chapter 9. Their outputs are fused into a `TaskGoalDraft` according to fixed precedence rules. The dialogue manager asks for missing information and presents the completed interpretation for confirmation. The resulting `TaskGoal` contains the task constraints; the planner determines the execution sequence later.

Configured colour names act as identity aliases. For example, the request *Move the green shuttle to slot 4 on the right rail.* resolves to `room315_right_shuttle_3`, the right rail and slot 4 before publication. A request such as *Move the nearest loaded right shuttle to slot 3* remains a selection policy until current state is available. It records the right rail, loaded-payload condition and nearest-selection rule without assigning an identity prematurely.

10.3 Step 2—read the current scene and ground the goal

The visual runtime synchronises the left and right RGB camera topics and runs the selected ResNet-18 checkpoint. It decodes the 200 predicted values only for identities marked as present in the configured registry. Before publishing a typed `VisualStateObservation`, it checks the artefact, timestamp, vocabulary and internal consistency. A current observation supplies planner facts only after these checks pass.

The task gateway combines the current visual observation and configured presence registry with supervisor device state and identity-bearing rail sensor readings to create an `ObservedState`. Planner localisation excludes the controller’s segment and longitudinal-position fields. Continuous controller position is reserved for the supervisor’s switch-proximity veto. Because removable external obstacles are disabled in the current configuration, only another shuttle can occupy and block a rail route.

The gateway validates and, when required, grounds the task against the current `ObservedState`. Explicit requests already identify a shuttle, whereas an *any*, *nearest* or station-slot policy is resolved from the current state and route occupancy. A loaded/empty payload condition requires five consecutive fresh visual frames by default before an identity is selected. The grounded task keeps the original goal identifier and remains fixed throughout its execution cycle.

10.4 Step 3—build a fresh PDDL problem and plan

10.4.1 From current state to decision predicates

For each decision cycle, the executive creates a complete Room 315 PDDL problem from the latest `ObservedState` and the grounded goal. The problem contains the present shuttles, their current symbolic locations, slot occupancy, payload facts, switch and stopper states, and the requested goal. The domain encodes only supported operations and rail-topology constraints.

The PDDL layer receives neither raw images nor free text. Its versioned domain defines typed parameters, preconditions, symbolic effects and a relative cost for every action. The generated problem contains the current known facts, grounded target and goal for the current cycle. Unknown, stale or conflicting state prevents problem generation and is never interpreted as an empty slot or a clear route.

10.4.2 How the current facts constrain the next action

For a transport task, each applicable action family requires `validated_state`, the matching `active_goal_side` and their own location, occupancy and route predicates. POPF searches for sequences whose preconditions can hold and whose effects establish the goal, while the generated

problem requests `minimize (total-cost)`. These symbolic costs favour plans with fewer unnecessary setup actions or relocations among legal alternatives. Travel-time modelling and supervisor safety checks remain separate. Table 10.1 summarises the principal cases, and the versioned domain contains the complete typed conditions.

Table 10.1: Principal symbolic decision cases in the current Room 315 PDDL domain.

Current symbolic situation	Decisive conditions	Applicable action and symbolic result
Canonical exterior-route preparation	Validated active side, connected stations, normal-route and named device-group facts	<code>prepare_switches</code> , then <code>open_stoppers</code> , establish <code>switches_ready</code> , <code>stoppers_open</code> and <code>path_ready</code> when those facts are still required
Direct move from an exact slot	Source occupancy agrees with the selected shuttle; target is <code>slot_free</code> ; route is clear and ready; no clearance is pending	<code>move_shuttle_to_slot</code> releases the source occupancy and marks the target occupied and reserved
Switch-specific or segment-origin route	A topology block is known; a route to the free target is available and clear; no clearance is pending	<code>prepare_topology_route</code> or <code>prepare_slot_topology_route</code> , followed by the matching topology move, configures a legal route without inventing a source slot
Another shuttle blocks the required route	An ordered clearance is pending; the next relocation or staging destination is capacity-safe and its interior entry is clear	The clearance family enters <code>clearance_mode</code> and selects one permitted next relocation. Each completed step produces a fresh problem; finish or pause actions control route restoration
A previous mixed route needs normalisation	Route reconfiguration is allowed; clearance mode is inactive; no relocation remains	<code>restore_normal_route</code> restores the exterior route and its readiness facts
Arrival or inspection is ready to close	Target station/slot and <code>DISABLED</code> controller-state facts hold, or a confirmed inspection is required	<code>stop_shuttle</code> and the terminal actions establish task completion; <code>inspect_state</code> completes an inspection without an actuation command

PDDL effects describe the state expected after an action. The next planning cycle nevertheless uses the newly observed state rather than assuming that the prediction occurred. The complete B03 run later in this chapter shows this difference between a symbolic effect and a reported physical result. In the normal planning path, the executive sends the domain and problem to the PlanSys2 planner service, configured with the POPF solver. Planning uses the service directly rather than the PlanSys2 Executor. The executive then translates and validates the symbolic plan and selects its first supported `PddlPlanStep/TranslatedPlanStep` pair. Each executed decision is therefore tied to the state used to plan it.

10.5 Step 4—translate, supervise and move

The PDDL translator converts supported actions into `TranslatedPlanStep` records, each containing a sparse, named command dictionary. A switch command names only the switches that must change; a stopper command names only its target devices; and a shuttle command includes the identity, side, direction, speed or target slot required by that action. The auxiliary `event_action` field records the source action for dataset tooling, whereas the first step’s `command` field supplies the live request.

The translated request passes through the supervisor checks described in chapter 8. If those checks pass, the supervisor publishes a typed `SwitchCommand`, `StopperCommand` or `ShuttleCommand`. The kinematic controller consumes that message and changes the shuttle state and Gazebo pose.

10.6 Step 5—check the result and decide what follows

After executing the selected step, the executive obtains newer state reports and checks its action-specific postcondition. A switch or stopper action must match the requested controller state, while a motion action must establish the expected location or route effect. If the grounded goal remains incomplete, the next cycle builds a new PDDL problem from the observed after-state. Any change in occupancy is included explicitly when replanning.

Final-slot success is reported only when the DZI and controller conditions defined in chapter 8 are met. Otherwise, the executive obtains a newer observation and starts another planning cycle.

10.7 Case B03 from instruction to arrival

The recorded instruction was *Move the green shuttle to slot 4 on the right rail*. Table 10.2 follows the main stages of this successful cold-start execution. The campaign case defined the instruction, goal and launch state, then left the action sequence to the planner. In the recorded run, it selected one direct-motion step to R3’s slot 4 destination, which maps to DZI4R.

Table 10.2: English-to-motion record for campaign case B03.

Stage	Recorded result
Interactive request	<i>Move the green shuttle to slot 4 on the right rail</i> . The harness supplied <code>yes</code> after the rendered interpretation matched the case declaration
Confirmed task	Transport <code>room315_right_shuttle_3</code> on the right rail to slot 4
Current state	Visual state supplies current location and occupancy; a fresh PDDL problem is sent to POPF and the executive selects the first supported step
Execution step	The recorded initial state places R3 at slot 3 with slot 4 free. The motion portion therefore contains one <code>move_shuttle_to_slot</code> step; it requires neither blocker clearance nor a topology relocation
Motion command	<code>move_shuttle_to_slot</code> for R3 from right slot 3 to right slot 4 within the Staubli section at 0.2 m/s
Arrival	Two separately timestamped DZI4R readings report identity R3; the matching <code>ShuttleState</code> reports target slot 4 and <code>DISABLED</code> , and a newer visual observation records the final scene
Outcome	Every recorded postcondition is satisfied; the final status is <code>succeeded</code> with reason <code>task_goal_satisfied</code>

Selection policies such as *nearest loaded shuttle* use the same sequence, with the identity chosen from the current scene before the PDDL problem is built. The recorded explicit-identity, colour-alias and payload-qualified cases on both rails followed this workflow. Chapter 11 presents the campaign outcomes and visual-model measurements.

Chapter 11

Experiments and Results

11.1 Integrated English-to-motion campaign

11.1.1 Campaign design

The integrated campaign was defined before execution as a twelve-row test matrix. Each row fixed a case identifier, one unique English utterance, its predeclared case family and expected grounded goal, together with the rail, active identities, initial slot and payload arrangement. Slot requests fixed one terminal slot; station requests fixed the set of acceptable station slots. The eight B-series rows left their symbolic action sequence to the planner. Only the two P-series and two M-series route-clearance rows additionally fixed the exact four-step trace that execution had to reproduce. The saved environment record identifies the software, language configuration, visual checkpoint, runtime configuration and PDDL domain used for the campaign. The runner checked these files before starting the cases.

Every row used a cold start: a fresh Gazebo process and an isolated ROS domain, with no retained task, observation or controller state from another row. Before the utterance was entered, the runner waited for the world and scene, both rail controllers, both overhead camera streams, the configured visual runtime, the PlanSys2 planner service, the supervisor and the task gateway. The matrix assigned six rows to each rail, exercised all eight configured identities and partitioned the cases into five mutually exclusive families. Four families describe the language or goal form: colour alias, explicit identity, station target and payload-qualified nearest. The fifth is a route-clearance scenario family whose explicit-identity requests require a blocking shuttle to be staged first. Table 11.1 lists every declared row alongside the recorded motion and terminal result.

For each case, the runner entered the instruction, compared the displayed interpretation with the expected goal and then replied `yes` through the same interactive confirmation used in normal operation. The CLI's auto-confirm mode was not used. A row counted as successful only when the expected terminal state was reached, every step postcondition passed, the required commands passed the supervisor checks and the final DZI and controller readings agreed. The camera recording and ROS bag also had to close cleanly.

Table 11.1: Campaign rows and recorded terminal results.

Case	Case family	Rail / shuttle	Recorded task path	Steps / postconditions	Status
B01	Colour alias	Right / R1	Slot 1 → slot 2	1/1	Passed
B02	Explicit identity	Right / R2	Slot 2 → slot 3	1/1	Passed
B03	Colour alias	Right / R3	Slot 3 → slot 4	1/1	Passed
B04	Station target	Right / R4	Slot 4 → Yaskawa, slot 1	1/1	Passed
B05	Colour alias	Left / L1	Slot 1 → slot 2	1/1	Passed
B06	Explicit identity	Left / L2	Slot 2 → slot 3	1/1	Passed
B07	Colour alias	Left / L3	Slot 3 → slot 4	1/1	Passed
B08	Station target	Left / L4	Slot 4 → Yaskawa, slot 1	1/1	Passed
P01	Payload-qualified nearest	Right / R1	Loaded R1 selected; R2 cleared; slot 1 → slot 3	4/4	Passed
P02	Payload-qualified nearest	Left / L1	Loaded L1 selected; L2 cleared; slot 1 → slot 3	4/4	Passed
M01	Multi-shuttle route	Right / R1	R2 cleared; slot 1 → slot 3	4/4	Passed
M02	Multi-shuttle route	Left / L1	L2 cleared; slot 1 → slot 3	4/4	Passed

The twelve rows represent supported normal-workflow tasks in valid initial scenes. The automated suites in section 11.3 separately exercise stale state, sensor loss, emergency stop and protected-switch rejection.

11.1.2 Final campaign result

All 12 planned cases were completed: 12/12 runs succeeded, all 24 recorded step postconditions were satisfied (24/24), and all 48 recorded supervisor action decisions passed their checks (48/48). Across the twelve runs, the executive made 24 plan attempts and recorded four occupancy-triggered replan events; it recorded no unknown-result retries and issued no safe aborts. All tasks ended with `status=succeeded` and `reason task_goal_satisfied`. Every run met the criteria defined above. Together, the runs trace the complete path from twelve distinct English instructions to their expected shuttle movements in the selected Gazebo scenes.

Figure 11.1 compares the initial and final camera frames from case B03 and shows the corresponding change in the simulated scene.

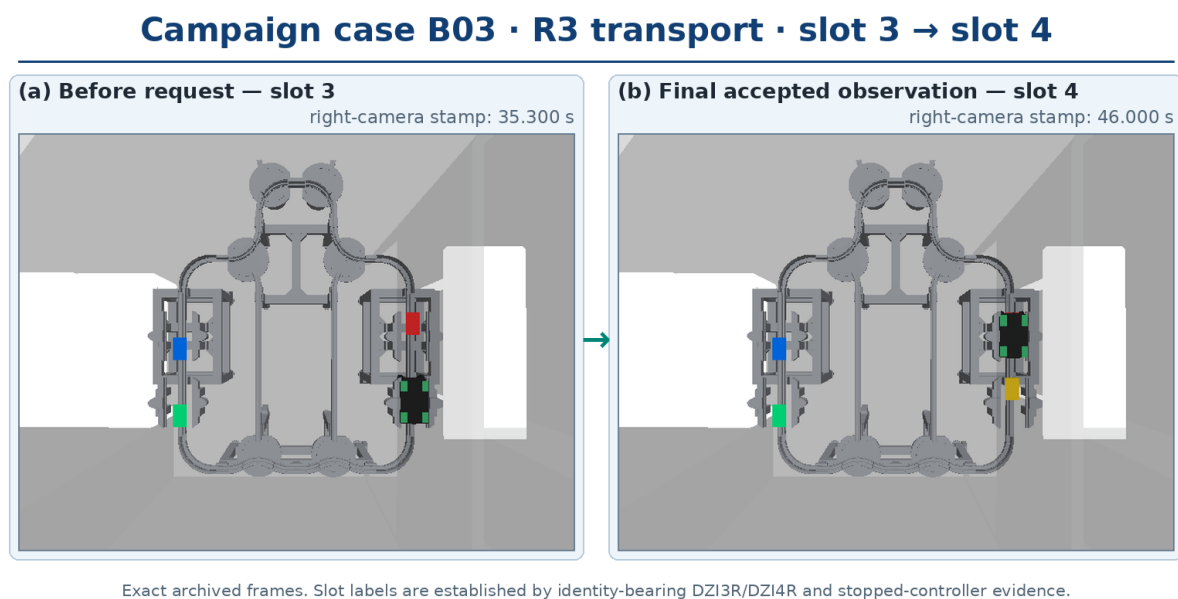


Figure 11.1: Initial and final right-camera frames from the B03 ROS bag. The frames show the scene change; identity-bearing DZI and controller records identify R3’s arrival at slot 4 and the final `DISABLED` controller state.

Figure 11.2 gives a compact view of the campaign outcomes and coverage.

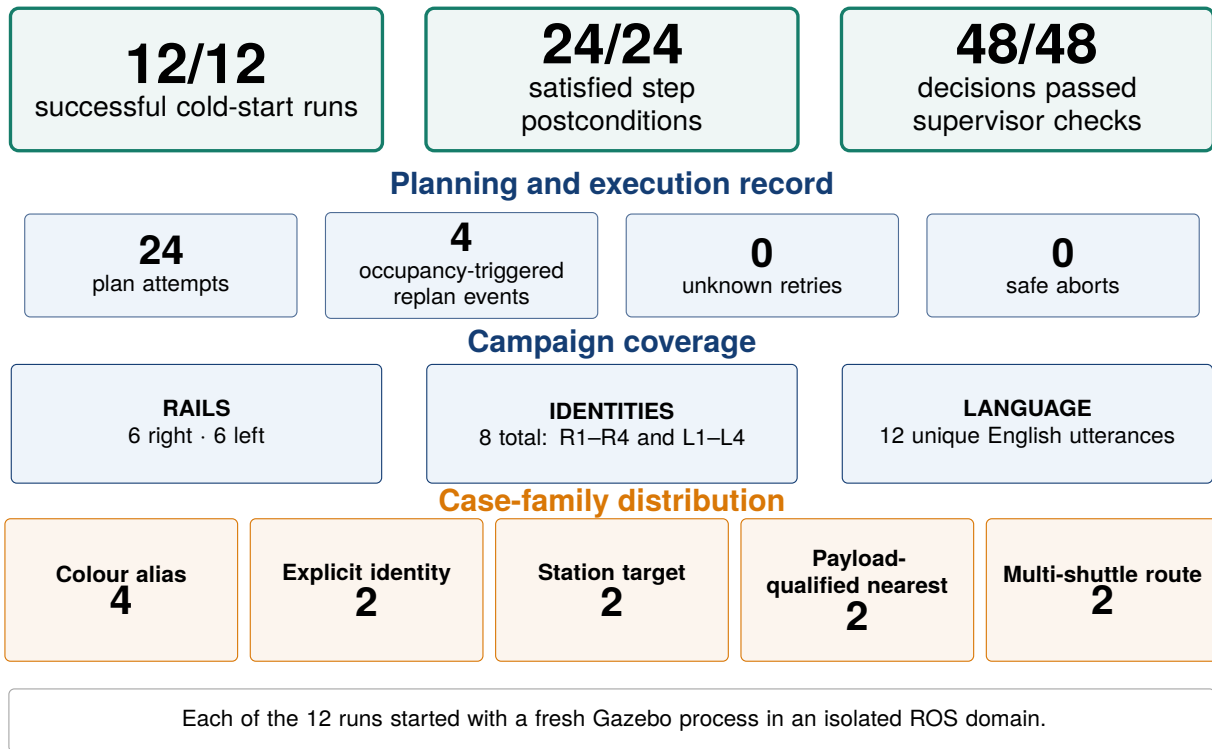


Figure 11.2: Outcome counts, planning record and planned coverage of the twelve-run campaign. The five case-family counts are mutually exclusive.

11.2 Visual model selection and locked-Test result

The visual model was trained and selected by the method in chapter 9. Epoch 14 minimised validation total weighted loss at 0.10294849 and produced the configured `best.pt`. After selection, that unchanged checkpoint was evaluated once on the locked 256-scenario Test partition. Table 11.2 reports its principal metrics.

Table 11.2: Locked-Test metrics for the configured visual-state checkpoint.

Metric	Result	Interpretation
Rail-side accuracy	100.00%	Correct left/right rail classification
Loaded-state accuracy	99.2537%	Correct loaded/empty state for present fixed identities
Exact block accuracy	65.2985%	Correct highest-scoring one of 14 symbolic blocks
Top-two block accuracy	74.8134%	Correct block among the two highest scores
Arc-position MAE, s_m	0.15152 m	Mean absolute longitudinal-position error
Normalised-position MAE, s_{ratio}	0.096266	Error relative to the known segment length
Bounding-box MAE	16.076 pixels	Mean absolute projected-box coordinate error

Rail-side and payload-state accuracy are high, while exact fine-grained block classification is less precise. The deployed model therefore supplies location and payload estimates to the workflow rather than deciding a final slot by itself. Exact arrival is taken from the rail-state checks defined in chapter 8; a later camera observation records the changed scene.

11.3 Automated software checks

Before the complete runs, automated tests checked the software used by the workflow. Static and unit checks cover geometry, configuration and schema rules. Component and controlled-integration tests exercise providers, task construction, planning, translation, supervision and effect checking in controlled harnesses. They also include fault and protective cases. The integrated campaign then evaluates the normal-path components together in the Gazebo workflow.

On 7 August, three named suites were executed. The focused contract and executive suite passed 190/190 checks in 4.10 s, including the Torch-dependent visual-runtime case through the campaign’s visual Python path. A transport suite passed 96/96 checks in 1.48 s, covering kinematic paths, topology, headway, reservations, fleet supervision, devices, identity and static shuttle clearance. The complementary operator-capability suite passed 42/42 checks in 0.41 s, covering the robot command helper, launch propagation, rail devices, marker/sensor lifecycle and switch-feedback rules. The suites overlap in a small number of files and are therefore reported separately rather than summed. Full commands and results are described in appendix A.

11.3.1 Language-interface evaluation

The nine non-control cases in the unchanged ten-case English corpus were each executed once through the configured local semantic gateway, strict envelope parser and field fusion; successfully parsed drafts were then passed to domain validation. The dialogue manager handled the cancellation case before model inference. All ten declared final outcomes matched: four supported goals produced the expected constraints, three incomplete or ambiguous requests required clarification, two inconsistent or invalid requests were rejected, and the cancellation command produced no task. All five incomplete, ambiguous, inconsistent or invalid requests ended in clarification or rejection. The configured local backend was invoked exactly once in all nine non-control cases. Eight outputs passed the strict semantic-envelope schema without fallback. For the invalid-slot case, the model output itself contained slot 9; the schema rejected that envelope and the request remained an error. The cancellation command bypassed inference. This ten-utterance study evaluates the implemented Room 315 English contract. The retained case records and configuration are described in appendix A.

11.3.2 All-robot launch check

A separate cold-start check launched the full-floor profile with `robots:=all` in an isolated ROS domain and Gazebo partition. All six configured robot models were present; each exposed its namespaced joint command and feedback topics and produced a fresh `JointState` sample, while both TIAGo variants exposed independent base-velocity topics. The active log before planned shutdown was free of the declared error patterns. This check covers launch composition, namespace separation and feedback availability. Motion, simultaneous operation and repeatability require dedicated runs.

Figure 11.3 places these results alongside the other evaluation layers. Software tests study local behaviour and protective cases, the offline evaluations measure the language and visual interfaces, and the campaign follows the complete normal workflow. Full evaluation records are indexed in appendix A; operator tools and runbooks are listed in appendix B.

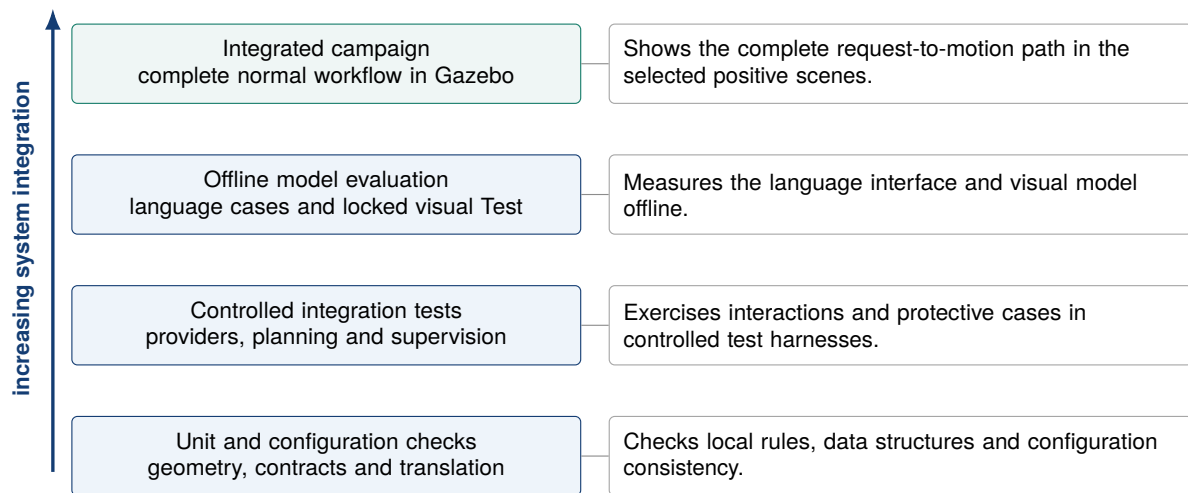


Figure 11.3: Software tests, offline model evaluation and complete simulated runs answer different questions about the project.

11.4 Review of the internship objectives

Table 11.3 returns to the O1–O4 objectives introduced in chapter 1 and summarises how each one was assessed.

Table 11.3: Assessment of the four internship objectives.

Objective	Main result	How it was evaluated
O1—simulation platform	Third-floor and Room 315 Gazebo models plus one all-robot cold-start check: six of six configured models, namespaced command/feedback topics and fresh state samples	One full-floor cold-start launch and interface check
O2—multi-shuttle transport	Calibrated Hermite rail geometry, explicit switch-dependent topology, eight stable shuttle identities, typed devices, reservations, headway checks and limited recovery on both rails; 96/96 named transport checks and route-clearance cases in the integrated campaign	Named software tests and declared simulated campaign scenes
O3—contracts and supervision	Versioned command, state and effect contracts; sparse device updates; freshness and consistency gates; supervisor rejection cases; and post-step effect checks covered by the 190/190 focused checks and the integrated runs	Named software tests and simulated campaign records
O4—neuro-symbolic workflow	Family-grouped paired-camera corpus, configured visual checkpoint and locked-Test metrics; ten-case language evaluation; and the twelve-run English-to-motion campaign	Ten language cases, the locked visual Test split and twelve selected campaign scenes

Chapter 12

Limitations and Perspectives

12.1 Current limitations

The integrated campaign and the language and perception studies concern the Room 315 Gazebo workflow. The separate all-robot check covers the full-floor Gazebo launch profile. The shuttle backend is kinematic, which supports controlled studies of topology, coordination and closed-loop task execution. It omits torque, slip, braking, mechanical play and payload inertia. Arm manipulation, handover and grasp verification are also beyond the implemented task domain. The supervisor provides software checks for this simulated workflow; it is not a certified machinery-safety function.

The perception study uses the known shuttle fleet, fixed simulated cameras and a grouped held-out split of generated scenes. Its measurements characterise that camera geometry and those scene families. The visual model estimates shuttle location and payload state, while the switch, stopper and actuator interfaces use their own reported state.

Each campaign row and the all-robot launch check were executed once. Repeating them across controlled seeds is the next step for measuring run-to-run variation. Automated suites exercise stale state, sensor loss, planner timeout, emergency stop and conflicting tasks; a live-graph fault campaign would extend that work. Final shuttle arrival already uses command-linked observations. Applying the same correlation to switch and stopper reports is a remaining interface improvement. The current deadlock rule handles crossed reservations; longer wait-for cycles and temporal non-progress require further detection.

The documented CLI validates a task against the Room 315 domain and obtains operator confirmation at the configured risk levels. The execution subscriber assumes a trusted ROS graph and the official CLI producer. A multi-user simulation service would therefore need authenticated task sources or graph isolation, together with sender identity in each task.

The language study covers ten selected English requests, including clarification, rejection and cancellation. Open-vocabulary and multilingual evaluation remain future extensions.

12.2 Next evaluation steps

Table 12.1 orders the next simulation-focused tasks by the question each one would answer.

Table 12.1: Priority experiments for extending the current evaluation.

Priority	Engineering task	Purpose
P0	Repeat the campaign across controlled seeds	Measure repeatability, seed sensitivity and uncertainty, with every attempt recorded in a predefined matrix
P0	Inject faults into the live ROS graph	Characterise shutdown and recovery for stale or missing visual state, sensor loss, planner timeout, emergency stop and conflicting tasks
P0	Relate switch and stopper acknowledgements to commands	Correlate each device result with a matching state update newer than the command
P0	Authenticate task sources and retain operator confirmation	Add approved-source identity and keep confirmation at the appropriate risk levels
P1	Package one reproducible evaluation release	Keep together the software, fixtures, data and commands needed to repeat the tests, offline metrics and campaign

12.3 Continued use at MFJA

The current package can support practical teaching sessions in which students launch a chosen robot set, inspect namespaces and typed interfaces, modify a rail scenario, generate a PDDL problem and follow one supervised task step to its observed result. Configuration-driven launch and cold-start campaign rows allow every session to begin from a declared state, while the runbooks provide the operational sequence and diagnostic checks.

The same assets provide a reference benchmark setup. A new planner, perception checkpoint or task-grounding method can be compared on the existing case matrix without changing controller behaviour or the arrival checks. The preserved 12-case campaign supplies a reference scenario set, and the grouped visual corpus supplies a fixed split for model comparison.

Further synthetic-data work can add camera perturbations, partial occlusion, station clutter and payload variation while preserving configuration-family grouping and a locked Test partition. This would broaden the visual study while preserving comparability with the current data and model records.

12.4 Focused research extensions

Planner development should keep the PDDL domain aligned with the canonical rail graph and require an executable translation and visible effect check for every action. Independently tested atomic tasks can then be extended to controlled concurrency, with reservations and deadlock handling evaluated before any increase in parallelism.

The language interface can be extended with clearer explanations for rejected or ambiguous requests and a recorded correction history. New language or vision methods can then be compared with the current implementation on the existing case matrix.

Chapter 13

Conclusion

The four objectives were completed in simulation. Objectives O1 and O2 produced a maintainable ROS 2 Jazzy and Gazebo Harmonic environment with selective robot launch, calibrated rail cells, identifiable shuttles and switch-dependent routes. One all-robot cold start confirmed access to the six configured models, and all 96 transport checks passed across the geometry, topology, coordination, device and identity functions.

Objectives O3 and O4 extended this foundation with typed command and state interfaces, task planning, language interpretation and paired-camera perception. Together, these elements formed the evaluated English-to-motion workflow.

The language interface also produced the intended outcome in all ten evaluation cases, including clarification, rejection and cancellation.

The complete path from an English request to shuttle motion was exercised in 12 isolated cold-start runs. Every run reached its intended terminal state, and all 24 recorded step postconditions were satisfied. On the locked 256-scenario Test set, the configured visual checkpoint achieved 100% rail-side accuracy, 99.2537% loaded-state accuracy and 65.2985% exact-block accuracy.

For MFJA, the project provides a reusable platform for practical teaching, software integration, synthetic-data generation and further planning or perception experiments. For me, the most important lesson was learning how to divide an initially broad integration request into smaller parts that I could implement and test one at a time. I also prepared the configuration, regression tests and run instructions so that the host team can continue using and extending the work.

The immediate next step is to repeat the campaign across controlled seeds and conduct a live-graph fault-injection study before extending the simulated workflow to broader scenarios.

Bibliography

- [1] AIP-PRIMECA Occitanie. MFJA third-floor Gazebo and Room 315 robotics stack. https://github.com/aip-primeca-occitanie/mfja_3rd_floor_gz, 2026. Project repository, accessed 6 August 2026.
- [2] Amanda Jane Coles, Andrew I. Coles, Maria Fox, and Derek Long. Forward-chaining partial-order planning. In *Proceedings of the Twentieth International Conference on Automated Planning and Scheduling*, pages 42–49, 2010. doi: 10.1609/icaps.v20i1.13403.
- [3] Timothy Duggan, Pierrick Lorang, Hong Lu, and Matthias Scheutz. The price is not right: Neuro-symbolic methods outperform VLAs on structured long-horizon manipulation tasks with significantly lower energy consumption. *arXiv preprint arXiv:2602.19260*, 2026. doi: 10.48550/arXiv.2602.19260. URL <https://arxiv.org/abs/2602.19260>. Accepted at the 2026 IEEE International Conference on Robotics and Automation (ICRA).
- [4] Georgi Gerganov and llama.cpp contributors. llama.cpp: LLM inference in C/C++. Software repository, 2023. URL <https://github.com/ggml-org/llama.cpp>. Accessed 7 August 2026.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- [6] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, et al. Do As I Can, Not As I Say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023. URL <https://proceedings.mlr.press/v205/ichter23a.html>.
- [7] INSEE. NAF rev. 2, subclass 85.42Z: Higher education. <https://www.insee.fr/fr/metadonnees/nafr2/sousClasse/85.42z>, 2025. Accessed 4 August 2026.
- [8] International Organization for Standardization. ISO 23247-1:2021 automation systems and integration—digital twin framework for manufacturing—part 1: Overview and general principles, October 2021. URL <https://www.iso.org/standard/75066.html>.
- [9] Nishanth Kumar, William Shen, Fabio Ramos, Dieter Fox, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Caelan Reed Garrett. Open-world task and motion planning via vision-language model generated constraints. *IEEE Robotics and Automation Letters*, 11(3):3366–3373, 2026. doi: 10.1109/LRA.2026.3656799. URL <https://arxiv.org/abs/2411.08253>.
- [10] Francisco Martín, Jonatan Ginés Clavero, Vicente Matellán, and Francisco J. Rodríguez. PlanSys2: A planning system framework for ROS 2. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9742–9749, 2021. doi: 10.1109/IROS51168.2021.9636544.
- [11] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL—the planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, October 1998. URL <https://ipc08.icaps-conference.org/deterministic/data/mcdermott-et-al-tr-1998.pdf>. Version 1.2.
- [12] Open Robotics. Interfaces (topics, services, actions)—ROS 2 Jazzy documentation. <https://docs.ros.org/en/jazzy/Concepts/Basic/Interfaces-Topics-Services-Actions.html>, 2026. Accessed 5 August 2026.
- [13] Open Robotics. Quality of service settings—ROS 2 Jazzy documentation. <https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html>, 2026. Accessed 4 August 2026.
- [14] Open Source Robotics Foundation. Getting started with Gazebo Harmonic. <https://gazebo-sim.org/docs/harmonic/getstarted/>, 2026. Accessed 4 August 2026.
- [15] Open Source Robotics Foundation. Use ROS 2 to interact with Gazebo. https://gazebo-sim.org/docs/harmonic/ros2_integration/, 2026. Accessed 4 August 2026.
- [16] Open Source Robotics Foundation. SDF format specification. <https://sdformat.org/spec/>, 2026. Version 1.12; accessed 5 August 2026.

- [17] République française. API Recherche d'Entreprises: Université de Toulouse, SIREN 938 271 392. <https://recherche-entreprises.api.gouv.fr/search?q=938271392>, 2026. Official business-register record; accessed 5 August 2026.
- [18] Université de Toulouse. La Maison de la Formation Jacqueline Auriol (MFJA). <https://www.univ-tlse3.fr/lieux-de-ressources/la-maison-de-la-formation-jacqueline-auriol-mfja>, 2026. Accessed 4 August 2026.
- [19] Université de Toulouse. Schéma de promotion des achats socialement et écologiquement responsables 2026–2029. https://www.univ-tlse3.fr/medias/fichier/2026-03-ca-083_1773392372102-pdf, 2026. Institutional figures on p. 5; document approved 13 March 2026; accessed 4 August 2026.
- [20] Université d'Évry Paris-Saclay. Évaluation de la formation en entreprise (Module Stage M2): Le rapport. Internal university guidance, 2026. Official internship-report instructions, June 2026, p. 17, Section V: use of generative artificial intelligence.
- [21] Université d'Évry Paris-Saclay and Université de Toulouse. Internship agreement: Development of a unified interface for industrial and mobile robots in simulation, 2026. Signed internal document, p. 1, mission and activities; 2 March–28 August 2026.
- [22] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, Yaxin Peng, Feifei Feng, and Jian Tang. TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 10(4): 3988–3995, 2025. doi: 10.1109/LRA.2025.3544909. URL <https://arxiv.org/abs/2409.12514>.
- [23] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115. URL <https://arxiv.org/abs/2412.15115>.
- [24] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>.

Chapter A

Reproducibility Records

A.1 Software environment and handover

The reproducibility package identifies the final implementation and the records used for its evaluation. The handover includes an archived or committed copy of the exact source, together with the current configuration and report bundle [1]. The integrated campaign is retained in two layers: its lightweight records are versioned with the report, and its complete raw bundle is published as a checksum-identified release asset. The summary, file manifest and full-archive checksum list identify the workflow reported in chapter 11.

The reference environment is Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic and Python 3.12. Manifests and hashes provide stable identifiers for generated data and model results. The report sources and lightweight evidence are versioned under `report/` in the software repository and are also included in the submission bundle.

The workspace can be identified and built from its root with:

```
1 source /opt/ros/jazzy/setup.bash
2 git -C src/mfja_3rd_floor_gz rev-parse HEAD
3 colcon build --symlink-install --base-paths src/mfja_3rd_floor_gz
4 source install/setup.bash
```

Source and report checks. The current source identity and test inventory can be recorded from the repository root with:

```
1 git rev-parse HEAD
2 git status --short
3 python3 -m pytest --collect-only -q \
4   mfja_robot_control_config/test \
5   mfja_3rd_floor_description/test
```

A reproducible handover records this output and archives or commits the same source. Test collection confirms discovery, while the executed results are presented in chapter 11. From `report/`, `make check` builds the PDF, rejects unresolved references or citations and reports the page size and count. Its diagnostics are preserved as `report/evidence/final_report_build_2026-08-08.log`.

User-facing launch profiles and their current arguments are documented in `docs/INSTALLATION.md`, `docs/QUICK_START_AND_FEATURE_GUIDE.md` and `docs/ROOM315_LANGUAGE_TO_MOTION_RUNTIME.md`. The browser-based `runbook.html` supplies copy-ready operator workflows and diagnostic checks. Appendix B provides a concise index of the corresponding capabilities and operator interfaces.

A.2 Evaluation files and identifiers

Table A.1 associates each reported result with the files and identifiers needed to inspect it, without depending on host-specific storage paths.

Table A.1: Files and identifiers for the reported results.

Result group	Files and identifiers
Software and configuration	Git HEAD c44c501640e3dab35ab241384c99963e1b9d8745, the stored binary worktree patch, and the 483-file source manifest with aggregate SHA-256 326ccd537a65c43bca5780610d4fb9a0d7bd95791b047f3a1e174fc481f889f9. The two untracked files named by the source record are retained under report/evidence/current_source_supplement_2026-08-07.
Configured language runtime	Qwen2.5-1.5B-Instruct-GGUF, qwen2.5-1.5b-instruct-q4_k_m.gguf, SHA-256 6a1a2eb6d15622bf3c96857206351ba97e1af16c30d7a74ee38970e434e9407e. The campaign used task_goal_understanding.local.yaml, SHA-256 6d8c8658aa03273c45d738f8fd7437bf516713ebca47cb508fc1b999d07d35ea; it pins the offline llama.cpp parameters and prompt-schema version.
Language-contract evaluation	report/evidence/room315_language_semantic_evaluation_v3 retains the unchanged ten-case corpus, semantic configuration, runner, source snapshot, raw backend outputs, strict envelopes, fusion traces and environment. The summary, manifest and checksum-list SHA-256 values are respectively 38d015617856e3127da5a7082701caf5ab40523c6271cc7830d521800c8ecffd, d6584fb276f12630c60bb2d3860988d06d0c63d2a0c2e2931f38aaf33c0574de and f3e4260f6514f35486783c22b74b932de264ad7af9b18c85e6868fdea859211b. The final protocol records 10/10 declared outcomes, nine model invocations, eight strict envelopes without fallback, one expected schema rejection and no unsafe automatic resolution.
Configured visual runtime and evaluation	Checkpoint SHA-256 8a2d865e3d3551ec4284b53aa913d66f24640e23556f2f26b49a165f3ce8d51d, bound to the preserved training_config.json, metrics.json, run_metadata.json, target/vectoriser sidecars and runtime preflight. visual_state_runtime.yaml selects this hash and has SHA-256 e96b4eb4c36e6b3f45a45a3fff3b8e3bf53ddd50fe3e603ddb1d402ee39e7c19. The evidence root is report/evidence/visual_model_evaluation_2026-07-30; its checksum-list SHA-256 is 912438606d432c8acc48bbf20aa624c65c29ea51ddef03142d75d2259c13c3b9.
Training and validation data	The preserved split_manifest.json (SHA-256 8516f38ffbfdfb5a70cab17bb480e31f4c1f2b0d4c360b9d91a0bf10c122ca93) records 1528 training, 256 validation and 256 locked-Test scenarios. The leakage audit (SHA-256 205e1d8f47109aa17b936190b86c66d3e7c6b0fbf49116c9477f75d1eeb97a03) and final review (SHA-256 aa21e1991a1ffac6070231cae25a880528bf4bc6e19b27f39e450483c1f9e7e0) retain the split and Test-evaluation checks.

Result group	Files and identifiers
Automated verification	Three logs retain each command, result and the same source identity as the campaign: pytest_current_source_focused_2026-08-07.log, 190/190, SHA-256 cf4804917505d61465559924b3c776dbc7e2c3418dfeec16bf8bc2de11a17110; pytest_current_source_transport_2026-08-07.log, 96/96, SHA-256 9bb7bf0448f323c87b16b9bb0eaca4fef74b6bfdf53473af084aaedf45ba1404; and pytest_current_source_runbook_capabilities_2026-08-07.log, 42/42, SHA-256 fbd5891ec92e20a6b3a43528bdecfc0ff6f4554253ba34bbb181a6910ea6347c. Their coverage is described in chapter 11.
Multi-robot cold start	report/evidence/multi_robot_cold_start_smoke_2026-08-07 retains the predeclared protocol, active and complete logs, ROS graph, model list and six fresh state samples. result.json has SHA-256 59b60b1fddf37b74b86013dce195438259f41933db335aa0ccff85e83e014fae; the checksum-list SHA-256 is 5c3d60f5d6324eafb989b577369b50f686e8c8d62d37a8bf27008cfff384d8c0.
Integrated runtime	The lightweight evidence root is report/evidence/room315_integrated_campaign_v2. Its summary.json has SHA-256 3c065872051d0e26b5425b64132c3a6f4f1ab8b79b3088fda5d9894e721f0dle, and manifest.json has SHA-256 3a381517e2c3bf33d575606caf871f07b5fbe7ccf307e760fc77a7c1ba47e318, while the complete bundle with per-case ROS bags is indexed by ROOM315_INTEGRATED_CAMPAGN_ARCHIVE.md and published under room315-evidence-v2; its archive SHA-256 is 7775885867f8fde6c23d24e881d339447d5fc622be0143f2be5ca21d7f259b1b. The summary records 12/12 successful cold-start runs across the 12 declared positive cases: six on each rail, all eight shuttle identities and five predeclared case families.
B03 camera figure	The exact archived frames, extraction script and provenance are retained under report/evidence/room315_campaign_figures_2026-08-07. The figure SHA-256 is c1272484031885f0ec32334eec343fb284d52588f7b198429e7718a0af26551c; the derivative checksum-list SHA-256 is 466772cfd0ab4dc4e57a6cc261b3ca507d2cc6c09882f2755d10f117e76bb502.

The repository retains the declarations, readiness records, resolved commands, runtime logs and terminal verification. The release adds the twelve raw ROS bags and the original checksum list; a separate checksum list verifies the 929 retained files. The campaign environment records the Git and runtime identities and isolated ROS domains. The current-source supplement preserves the two test files required to reconstruct the recorded 483-file source state.

Chapter B

Capability and Interface Reference

This appendix provides a concise reference to the current operator tools and public interfaces. Full operating procedures remain in `runbook.html`, while chapter 11 reports the configurations exercised in the experiments.

B.1 Operator capability index

Table B.1: Current capabilities represented in the operator runbook.

Runbook area	Available interface or profile	Use
Room 315 and world profiles	Focused Room 315; full third floor with <code>robots:=none</code> , an exact/comma-separated robot selection or <code>all</code> ; and an isolated fixed-base robot with its table	Launch compositions for exercising integration boundaries separately or together
Runtime presentation	GUI or headless execution and paused/running start; light/full GUI choices on the Room 315 and full-floor paths; optional device markers	Inspection and resource-control options; markers support debugging and calibration and can be hidden during image capture
Robot inventory and control	Four fixed-base arms, full TIAGo and base-only TIAGo from the current full-floor registry; namespaced joint command/state surfaces and independent TIAGo base velocity	The helper checks profiles, vector length and units; collision-free trajectories require a separate planner
Mirrored rail setup	Independent left/right enablement, shuttle count, start slots, stored speed setting and initial enabled state	Current low-level simulator configuration. Supervised tasks retain the fixed L1–L4/R1–R4 identity domain
Shuttle lifecycle	Typed add service; per-entity ON, OFF, RESET and REMOVE; rail-wide ALL ON, OFF and RESET; continuous state feedback	Additional names support commissioning; supervised tasks use the fixed L1–L4/R1–R4 identity set
Devices and sensors	Per-rail switch and stopper command/state topics plus continuous aggregate binary readings for station, branch and stopper-linked detectors	Raw command publication for low-level diagnosis; closed-loop tasks reach the same devices through translation and supervision
Geometry maintenance	Guarded dry-run projection from Gazebo XYZ to a rail segment and <code>s_ratio</code> , with YAML updates available for slots, ordinary sensors and single-/multi-point stoppers	Offline maintenance utility. A configuration write requires relaunch and subsequent marker/feedback checking
Runtime diagnosis	ROS graph, topic, service and interface inspection together with build/source procedures	Support for reproducibility and troubleshooting

B.2 Public rail interfaces

The `mfja_rail_interfaces` package is independent of Gazebo-specific controllers, allowing simulator nodes, tests and alternative actuation providers to share public types.

Table B.2: Condensed rail-interface reference.

Interface	Principal semantics
NamedState	Canonical device name and symbolic state, embedded in switch/stopper arrays
Switch command/state	Partial named command and reported switch state
Stopper command/state	Partial named command and reported stopper state
ShuttleCommand	Named ON/OFF/RESET/REMOVE, start slot, stored speed setting and optional target slot
ShuttleState	Identity, mode, controller segment/arc length, pose, stored speed and reached target; these fields support runtime checks
SensorFeedback	Binary occupancy readings with detected identity, segment and calibrated coordinates
VisualShuttleState	Fused identity/presence validity plus learned side, block, box, s_m , ratio, segment length and loaded state
VisualStateObservation	Image stamps, schema/checkpoint, readiness, validation reasons, provenance and eight shuttle records
AddShuttle service	Requested identity/start state and explicit acceptance result
JointTrajectory JointState	Namespaced robot joint command and feedback; the operator helper validates profile, vector length and angular unit conversion
Twist/odometry/TF	Independent TIAGo mobile-base command and feedback surfaces for the full and base-only variants

Three fields require careful interpretation. `ShuttleState.speed` stores a speed setting rather than instantaneous velocity. `reached_target_slot` indicates low-level setpoint completion; the full task effect requires additional evidence. `VisualStateObservation.accepted` indicates that the structural and freshness checks passed, while individual learned values remain estimates.

B.3 Canonical assets

Table B.3: Canonical Room 315 mapping.

Rail	Shuttles	Station slots
Left	L1–L4	1–2 Yaskawa HC10; 3–4 KUKA KR6
Right	R1–R4	1–2 Yaskawa HC10DT; 3–4 Stäubli TX2

Rail geometry, public segment names, switch assignments and branch-capacity rules are defined and justified in chapter 7. Runtime topic strings remain configurable; the installed configuration and live ROS graph are the current operational references.

B.4 Runtime responsibility summary

Table B.4: Responsibilities in the Room 315 task workflow.

Workflow decision	Responsible component
Interpret an English request	The local model proposes semantic fields; explicit parsing, field fusion, domain validation and dialogue produce the confirmed <code>TaskGoal</code>
Estimate visible shuttle state	The paired-camera model estimates location and payload state for identities supplied by the configured presence registry
Create the symbolic plan	PlanSys2/POPF solves the current PDDL problem
Select the next operation	The project executive takes the first supported plan step and observes the result before replanning
Translate the operation	The plan translator creates the sparse named command for the addressed device or shuttle
Publish the command	The software supervisor checks the current state and publishes the corresponding typed ROS 2 message
Apply and confirm the result	The kinematic controller applies the simulated transition; the executive compares device and sensor reports with the expected effect

Canonical source roots and artefact identities are listed in appendix A.